

SOLIDWORKS API (.NET VBA C#) — Examples (25/50):

Get_Centerlines_in_Drawing_Example_CShar-

Get_Core_Feature_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Get Centerlines in Drawing Example (C#)

This example shows how to get all of the centerlines in all of the drawing views in a drawing.

```
//-----
// Preconditions:
// 1. Verify that the drawing document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing.
// 2. Inserts a centerline annotation.
// 3. Prints the path and file name of the drawing document
//    to the Immediate window.
// 4. Iterates the sheet and drawing view, prints their names, and
//    prints the name of the centerline annotation to
//    the Immediate window.
// 5. Examine the Immediate window.
//
// NOTE: Because this drawing document is used elsewhere,
// do not save any changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CenterLinesCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DrawingDoc swDrawing = default(DrawingDoc);
            View swView = default(View);
            CenterLine swCenterLine = default(CenterLine);
            Annotation swAnnotation = default(Annotation);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cylinder20.SLDDRW";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            status = swDrawing.ActivateView("Drawing View1");
            status = swModelDocExt.SelectByID2("cylinder20-9@Drawing View1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 0.513454307125032, 0.454946591641617, 250.013794595267, false, 0, null, 0);

            swCenterLine = (CenterLine)swDrawing.InsertCenterLine2();
            swModel.ClearSelection2(true);

            swView = (View)swDrawing.GetFirstView();
            Debug.Print("File = " + swModel.GetPathName());

            while ((swView != null))
            {
                Debug.Print(" View = " + swView.GetName2());
                swCenterLine = (CenterLine)swView.GetFirstCenterLine();
                while ((swCenterLine != null))
                {
                    swAnnotation = (Annotation)swCenterLine.GetAnnotation();
                    Debug.Print(" Name = " + swAnnotation.GetName());
                    swCenterLine = swCenterLine.GetNext();
                }
                swView = (View)swView.GetNextView();
            }

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>
            public SldWorks swApp;
        }
    }
}
```

Get Centerlines in Drawing Example (VBA)

This example shows how to get all of the centerlines in all of the drawing views in a drawing.

```
'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Inserts a centerline annotation.
' 3. Prints the path and file name of the drawing document
'    to the Immediate window.
' 4. Iterates the sheet and drawing view, prints their names, and
'    prints the name of the centerline annotation to
'    the Immediate window.
' 5. Examine the Immediate window.
'
' NOTE: Because this drawing document is used elsewhere,
' do not save any changes.
'-----
```

Option Explicit

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swDrawing As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim swCenterLine As SldWorks.Centerline
    Dim swAnnotation As SldWorks.Annotation
    Dim status As Boolean
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String

    Set swApp = CreateObject("SldWorks.Application")
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20.SLDDRW"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension

    status = swDrawing.ActivateView("Drawing View1")
    status = swModelDocExt.SelectByID2("cylinder20-9@Drawing View1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", 0.513454307125032, 0.454946591641617, 250.013794595267, False, 0, Nothing, 0)

    Set swCenterLine = swDrawing.InsertCenterLine2()
    swModel.ClearSelection2 True

    Set swView = swDrawing.GetFirstView
    Debug.Print "File = " & swModel.GetPathName

    Do While Not swView Is Nothing
        Debug.Print " View = " + swView.GetName2
        Set swCenterLine = swView.GetFirstCenterLine
        Do While Not swCenterLine Is Nothing
            Set swAnnotation = swCenterLine.GetAnnotation
            Debug.Print " Name = " & swAnnotation.GetName
            Set swCenterLine = swCenterLine.GetNext
        Loop
        Set swView = swView.GetNextView
    Loop

End Sub
```

Get Centerlines in Drawing Example (VB.NET)

This example shows how to get all of the centerlines in all of the drawing views in a drawing.

```

-----
Preconditions:
1. Verify that the drawing document to open exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified drawing.
2. Inserts a centerline annotation.
3. Prints the path and file name of the drawing document
   to the Immediate window.
4. Iterates the sheet and drawing view, prints their names, and
   prints the name of the centerline annotation to
   the Immediate window.
5. Examine the Immediate window.
NOTE: Because this drawing document is used elsewhere,
do not save any changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim swView As View
        Dim swCenterLine As CenterLine
        Dim swAnnotation As Annotation
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cylinder20.SLDDRW"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawing = swModel
        swModelDocExt = swModel.Extension

        status = swDrawing.ActivateView("Drawing View1")
        status = swModelDocExt.SelectByID2("cylinder20-9@Drawing View1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.513454387125032, 0.454946591641617, 250.013794595267, False, 0, Nothing, 0)

        swCenterLine = swDrawing.InsertCenterLine2()
        swModel.ClearSelection2(True)

        swView = swDrawing.GetFirstView
        Debug.Print("File = " & swModel.GetPathName)

        Do While Not swView Is Nothing
            Debug.Print(" View = " & swView.GetName2)
            swCenterLine = swView.GetFirstCenterLine
            Do While Not swCenterLine Is Nothing
                swAnnotation = swCenterLine.GetAnnotation
                Debug.Print(" Name = " & swAnnotation.GetName)
                swCenterLine = swCenterLine.GetNext
            Loop
            swView = swView.GetNextView
        Loop

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Centers of Mass in Drawing Views Example (C#)

This example shows how to get the centers of mass in drawing views.

```
//-----
// Preconditions: Open a drawing that has one or more centers of mass.
//
// Postconditions: Inspect the Immediate window for the coordinates of
// all of the centers of mass in all of the views of the
// drawing.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CenterOfMass_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDrawDoc;
        View swView;
        CenterOfMass swCenterOfMass;
        Annotation swAnnotation;
        int sheetCount;
        int viewCount;
        object[] ss;
        object[] vv;
        double[] coord;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawDoc = (DrawingDoc)swModel;

            viewCount = swDrawDoc.GetViewCount();
            ss = (object[])swDrawDoc.GetViews();

            for (sheetCount = ss.GetLowerBound(0); sheetCount <= ss.GetUpperBound(0); sheetCount++)
            {
                vv = (object[])ss[sheetCount];
                for (viewCount = 1; viewCount <= vv.GetUpperBound(0); viewCount++)
                {
                    Debug.Print(((View)vv[viewCount]).GetName2());
                    swView = (View)vv[viewCount];
                    swCenterOfMass = (CenterOfMass)swView.GetFirstCenterOfMass();
                    if ((swCenterOfMass != null))
                    {
                        swAnnotation = (Annotation)swCenterOfMass.GetAnnotation();
                        if ((swAnnotation != null))
                        {
                            Debug.Print(" Annotation name: " + swAnnotation.GetName());
                        }
                        coord = (double[])swCenterOfMass.GetCoordinates();
                        Debug.Print(" Center of mass coordinates: X: " + coord[0] + ", Y: " + coord[1] + ", and Z: " + coord[2]);

                        while ((swCenterOfMass.GetNext() != null))
                        {
                            swCenterOfMass = (CenterOfMass)swCenterOfMass.GetNext();
                            if ((swCenterOfMass != null))
                            {
                                swAnnotation = (Annotation)swCenterOfMass.GetAnnotation();
                                if ((swAnnotation != null))
                                {
                                    Debug.Print(" Annotation name: " + swAnnotation.GetName());
                                }
                                coord = (double[])swCenterOfMass.GetCoordinates();
                                Debug.Print(" Center of mass coordinates: X: " + coord[0] + ", Y: " + coord[1] + ", and Z: " +
                                coord[2]);
                            }
                        }
                    }
                }
            }
        }

        public SldWorks swApp;
    }
}
```


Get Centers of Mass in Drawing Views Example (VBA)

This example shows how to get the centers of mass in drawing views.

```
'-----
' Preconditions: Open a drawing that has one or more centers of mass.
'
' Postconditions: Inspect the Immediate window for the coordinates of
' all of the centers of mass in all of the views of the
' drawing.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawDoc As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swCenterOfMass As SldWorks.CenterOfMass
Dim swAnnotation As SldWorks.Annotation
Dim sheetCount As Long
Dim viewCount As Long
Dim ss As Variant
Dim vv As Variant
Dim coord As Variant
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swDrawDoc = swModel
```

```
viewCount = swDrawDoc.GetViewCount
ss = swDrawDoc.GetViews
```

```
For sheetCount = LBound(ss) To UBound(ss)
    vv = ss(sheetCount)
    For viewCount = 1 To UBound(vv)
        Debug.Print (vv(viewCount).GetName2())
        Set swView = vv(viewCount)
        Set swCenterOfMass = swView.GetFirstCenterOfMass
        If Not swCenterOfMass Is Nothing Then
            Set swAnnotation = swCenterOfMass.GetAnnotation
            If Not swAnnotation Is Nothing Then
                Debug.Print " Annotation name: " & swAnnotation.GetName
            End If
            coord = swCenterOfMass.GetCoordinates
            Debug.Print " Center of mass coordinates: X: " & coord(0) & ", Y: " & coord(1) & ", and Z: " & coord(2)

            While Not (swCenterOfMass.GetNext Is Nothing)
                Set swCenterOfMass = swCenterOfMass.GetNext
                If Not swCenterOfMass Is Nothing Then
                    Set swAnnotation = swCenterOfMass.GetAnnotation
                    If Not swAnnotation Is Nothing Then
                        Debug.Print " Annotation name: " & swAnnotation.GetName
                    End If
                    coord = swCenterOfMass.GetCoordinates
                    Debug.Print " Center of mass coordinates: X: " & coord(0) & ", Y: " & coord(1) & ", and Z: " & coord(2)
                End If
            Wend
        End If
    Next viewCount
Next sheetCount
```

Next sheetCount

End Sub

Get Centers of Mass in Drawing Views Example (VB.NET)

This example shows how to get the centers of mass in drawing views.

```
'-----
' Preconditions: Open a drawing that has one or more centers of mass.
'
' Postconditions: Inspect the Immediate window for the coordinates of
' all of the centers of mass in all of the views of the
' drawing.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swDrawDoc As DrawingDoc
    Dim swView As View
    Dim swCenterOfMass As CenterOfMass
    Dim swAnnotation As Annotation
    Dim sheetCount As Integer
    Dim viewCount As Integer
    Dim ss As Object
    Dim vv As Object
    Dim coord As Object
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
        swDrawDoc = swModel
```

```
        viewCount = swDrawDoc.GetViewCount
        ss = swDrawDoc.GetViews
```

```
        For sheetCount = LBound(ss) To UBound(ss)
            vv = ss(sheetCount)
            For viewCount = 1 To UBound(vv)
                Debug.Print(vv(viewCount).GetName2())
                swView = vv(viewCount)
                swCenterOfMass = swView.GetFirstCenterOfMass
                If Not swCenterOfMass Is Nothing Then
                    swAnnotation = swCenterOfMass.GetAnnotation
                    If Not swAnnotation Is Nothing Then
                        Debug.Print(" Annotation name: " & swAnnotation.GetName)
                    End If
                    coord = swCenterOfMass.GetCoordinates
                    Debug.Print(" Center of mass coordinates: X: " & coord(0) & ", Y: " & coord(1) & ", and Z: " & coord(2))
```

```
                    While Not (swCenterOfMass.GetNext Is Nothing)
                        swCenterOfMass = swCenterOfMass.GetNext
                        If Not swCenterOfMass Is Nothing Then
                            swAnnotation = swCenterOfMass.GetAnnotation
                            If Not swAnnotation Is Nothing Then
                                Debug.Print(" Annotation name: " & swAnnotation.GetName)
                            End If
                            coord = swCenterOfMass.GetCoordinates
                            Debug.Print(" Center of mass coordinates: X: " & coord(0) & ", Y: " & coord(1) & ", and Z: " &
```

```
coord(2))
```

```
                        End If
                    End While
                End If
            Next viewCount
```

```
        Next sheetCount
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Chamfer Dimension Example (C#)

This example shows how to the values of a chamfer dimension.

```
//-----
// Preconditions:
// 1. Open a drawing that contains a chamfered part.
// 2. Click Tools > Dimension > Chamfer.
// 3. Select a chamfered edge, select one of the lead-in edges, and click
// the drawing to display and select the chamfer dimension.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Gets the chamfer dimensions.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            Dimension swDimension = default(Dimension);
            bool status = false;
            double length = 0;
            double angle = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Get chamfer dimensions
            swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, -1);
            swDimension = (Dimension)swDisplayDimension.GetDimension();
            status = swDimension.GetSystemChamferValues(ref length, ref angle);
            Debug.Print("Is selected dimension a chamfer dimension? " + status);
            //1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print("Angle = " + (angle * 57.3) + " degrees");
            Debug.Print("Length = " + length);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Chamfer Dimension Example (VBA)

This example shows how to the values of a chamfer dimension.

```
'-----  
' Preconditions:  
' 1. Open a drawing that contains a chamfered part.  
' 2. Click Tools > Dimension > Chamfer.  
' 3. Select a chamfered edge, select one of the lead-in edges, and click  
' the drawing to display and select the chamfer dimension.  
' 4. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the chamfer dimensions.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swDisplayDimension As SldWorks.DisplayDimension  
    Dim swDimension As SldWorks.Dimension  
    Dim status As Boolean  
    Dim length As Double  
    Dim angle As Double  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
  
    'Get chamfer dimensions  
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)  
    Set swDimension = swDisplayDimension.GetDimension  
    status = swDimension.GetSystemChamferValues(length, angle)  
    Debug.Print "Is selected dimension a chamfer dimension? " & status  
    '1 radian = 180°/p = 57.295779513° or approximately 57.3°  
    Debug.Print "Angle = " & (angle * 57.3) & " degrees"  
    Debug.Print "Length = " & length  
  
End Sub
```

Get Chamfer Dimension Example (VB.NET)

This example shows how to the values of a chamfer dimension.

```

-----
Preconditions:
1. Open a drawing that contains a chamfered part.
2. Click Tools > Dimension > Chamfer.
3. Select a chamfered edge, select one of the lead-in edges, and click
   the drawing to display and select the chamfer dimension.
4. Open the Immediate window.
Postconditions:
1. Gets the chamfer dimensions.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim swDimension As Dimension
        Dim status As Boolean
        Dim length As Double
        Dim angle As Double

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        'Get chamfer dimensions
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
        swDimension = swDisplayDimension.GetDimension
        status = swDimension.GetSystemChamferValues(length, angle)
        Debug.Print("Is selected dimension a chamfer dimension? " & status)
        '1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print("Angle = " & (angle * 57.3) & " degrees")
        Debug.Print("Length = " & length)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Chamfer Display Dimension Example (C#)

This example shows how to get chamfer display dimension properties. This example also shows how to set and get lower display dimension text.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\plate_tolistatus.sldprt.
// 2. Create a drawing from the part:
//    a. File > Make Drawing from Part.
//    b. Drag and drop the isometric view onto the sheet.
// 3. Create a chamfer display dimension.
//    a. Select Tools > Dimensions > Chamfer.
//    b. Select a chamfer edge.
//    c. Select a leading edge.
//    d. Click outside the part to place the display dimension.
//    e. Modify display dimension text in the PropertyManager:
//        1. In Dimension Text, click before <DIM>.
//        2. Select the diameter symbol.
//        3. Click after <DIM>.
//        4. Select the +/- symbol.
//        5. Type 0.5.
//        6. On the Other tab, select Override Units.
//        7. Click the green check mark to accept the
//           display dimension.
// 4. Open an Immediate Window.
// 5. Rename the namespace of this macro to match your C#
//    project name.
// 6. Run the macro.
//
// Postconditions: Inspect the Immediate Window and the
// display dimension in the graphics area.
//
// NOTE: Because the part document is used elsewhere,
// do not save any changes when closing it.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace ChamferDisplayDimension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        DisplayDimension swDispDim;
        bool bRet;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            bRet = swModel.Extension.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.4935677398765, 0.3280800260301, 0, false, 0,
            null, 0);

            swDispDim = (DisplayDimension)swSelMgr.GetSelectedObject6(1, 0);
            swModel.ClearSelection2(true);

            Debug.Print("Dimension type as defined in swDimensionType_e: " + swDispDim.Type2);
            //10=swChamferDimension
            Debug.Print("");
            Debug.Print("Uses document units? " + swDispDim.GetUseDocUnits());
            //false if uses local override units

            if (swDispDim.GetUseDocUnits() == false)
            {
                int LenUnit = 0;
                int AngUnit = 0;
                bRet = swDispDim.GetChamferUnits(out LenUnit, out AngUnit);
                Debug.Print("Uses local length unit as defined in swLengthUnit_e: " + LenUnit);
                //0=swMM
                //0=swDegrees, 3=swRadians
                Debug.Print("Uses local angle unit as defined in swAngleUnit_e: " + AngUnit);
            }
            Debug.Print("");

            int indx = 0;
            swDispDim.set_ChamferPrecision(indx, 0);
            Debug.Print("Precision of chamfer distance: " + indx + " decimal places");
            swDispDim.set_ChamferPrecision(indx, 1);
            Debug.Print("Precision of chamfer angle: " + indx + " decimal places");

            object objTokenFormats;
            object objTokenValues;
            string[] tokenformats = null;
            string[] tokenvalues = null;
            long n = 0;

            Debug.Print("");
        }
    }
}
```

```

Debug.Print("Text format items in " + swDispDim.GetNameForSelection() + ":");
Debug.Print("");
long count = 0;

count = swDispDim.GetTextFormatItems((int)swDimensionTextParts_e.swDimensionTextCalloutAbove, out objTokenFormats, out
objTokenValues);
Debug.Print("Number of callout above items: " + count);

if (!(count == 0))
{
    tokenformats = (string[])objTokenFormats;
    Debug.Print(" tokenformats: ");
    for (n = tokenformats.GetLowerBound(0); n <= tokenformats.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenformats[n]);
    }
    tokenvalues = (string[])objTokenValues;
    Debug.Print(" tokenvalues: ");
    for (n = tokenvalues.GetLowerBound(0); n <= tokenvalues.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenvalues[n]);
    }
}
Debug.Print("");
count = swDispDim.GetTextFormatItems((int)swDimensionTextParts_e.swDimensionTextPrefix, out objTokenFormats, out
objTokenValues);
Debug.Print("Number of prefix items: " + count);
if (!(count == 0))
{
    tokenformats = (string[])objTokenFormats;
    Debug.Print(" tokenformats: ");
    for (n = tokenformats.GetLowerBound(0); n <= tokenformats.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenformats[n]);
    }
    tokenvalues = (string[])objTokenValues;
    Debug.Print(" tokenvalues: ");
    for (n = tokenvalues.GetLowerBound(0); n <= tokenvalues.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenvalues[n]);
    }
}
Debug.Print("");
count = swDispDim.GetTextFormatItems((int)swDimensionTextParts_e.swDimensionTextSuffix, out objTokenFormats, out
objTokenValues);
Debug.Print("Number of suffix items: " + count);
if (!(count == 0))
{
    tokenformats = (string[])objTokenFormats;
    Debug.Print(" tokenformats: ");
    for (n = tokenformats.GetLowerBound(0); n <= tokenformats.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenformats[n]);
    }
    tokenvalues = (string[])objTokenValues;
    Debug.Print(" tokenvalues: ");
    for (n = tokenvalues.GetLowerBound(0); n <= tokenvalues.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenvalues[n]);
    }
}
Debug.Print("");
count = swDispDim.GetTextFormatItems((int)swDimensionTextParts_e.swDimensionTextCalloutBelow, out objTokenFormats, out
objTokenValues);
Debug.Print("Number of callout below items: " + count);
if (!(count == 0))
{
    tokenformats = (string[])objTokenFormats;
    Debug.Print(" tokenformats: ");
    for (n = tokenformats.GetLowerBound(0); n <= tokenformats.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenformats[n]);
    }
    tokenvalues = (string[])objTokenValues;
    Debug.Print(" tokenvalues: ");
    for (n = tokenvalues.GetLowerBound(0); n <= tokenvalues.GetUpperBound(0); n++)
    {
        Debug.Print(" " + tokenvalues[n]);
    }
}
}

//Create lower display dimension text and enclose
//in parentheses
swDispDim.SetLowerText("Example of lower text");
swDispDim.ShowLowerParenthesis = true;

```

```
Debug.Print("");
Debug.Print("Lower Dimension Text: " + swDispDim.GetLowerText());
Debug.Print(" Show lower parenthesis: " + swDispDim.ShowLowerParenthesis);
Debug.Print(" Show lower inspection: " + swDispDim.LowerInspection);
Debug.Print(" Split dual dimensions: " + swDispDim.Split);

}

public SldWorks swApp;

}

}
```

Get Chamfer Display Dimension Example (VBA)

This example shows how to get chamfer display dimension properties. This example also shows how to set and get lower display dimension text.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
' 2. Create a drawing from the part:
'   a. File > Make Drawing from Part.
'   b. Drag and drop the isometric view onto the sheet.
' 3. Create a chamfer display dimension.
'   a. Select Tools > Dimensions > Chamfer.
'   b. Select a chamfer edge.
'   c. Select a leading edge.
'   d. Click outside the part to place the display dimension.
'   e. Modify display dimension text in the PropertyManager:
'       1. In Dimension Text, click before <DIM>.
'       2. Select the diameter symbol.
'       3. Click after <DIM>.
'       4. Select the +/- symbol.
'       5. Type 0.5.
'       6. On the Other tab, select Override Units.
'       7. Click the green check mark to accept the
'          display dimension.
' 4. Open an Immediate Window.
' 5. Run the macro.

```

```

' Postconditions: Inspect the Immediate Window and
' the display dimension in the graphics area.

```

```

' NOTE: Because the part document is used elsewhere,
' do not save any changes when closing it.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDispDim As SldWorks.DisplayDimension
Dim bRet As Boolean

```

Sub main()

```

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swSelMgr = swModel.SelectionManager

    bRet = swModel.Extension.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.4935677398765, 0.3280800260301, 0, False, 0,
    Nothing, 0)
    Set swDispDim = swSelMgr.GetSelectedObject6(1, 0)
    swModel.ClearSelection2 True

    Debug.Print "Dimension type as defined in swDimensionType_e: " & swDispDim.Type2 '10=swChamferDimension
    Debug.Print ""
    Debug.Print "Uses document units? " & swDispDim.GetUseDocUnits 'false if uses local override units

    If swDispDim.GetUseDocUnits = False Then
        Dim LenUnit As Long
        Dim AngUnit As Long
        bRet = swDispDim.GetChamferUnits(LenUnit, AngUnit)
        Debug.Print "Uses local length unit as defined in swLengthUnit_e: " & LenUnit '0=swMM
        Debug.Print "Uses local angle unit as defined in swAngleUnit_e: " & AngUnit '0=swDegrees, 3=swRadians
    End If
    Debug.Print ""

    Dim indx As Long
    indx = swDispDim.ChamferPrecision(0)
    Debug.Print "Precision of chamfer distance: " & indx & " decimal places"
    indx = swDispDim.ChamferPrecision(1)
    Debug.Print "Precision of chamfer angle: " & indx & " decimal places"

    Dim tokenformats As Variant
    Dim tokenvalues As Variant
    Dim tokenformat As String
    Dim n As Long

    Debug.Print ""
    Debug.Print "Text format items in " & swDispDim.GetNameForSelection & ":"
    Debug.Print ""
    Dim count As Long

    count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextCalloutAbove, tokenformats, tokenvalues)
    Debug.Print "Number of callout above items: " & count

    If Not count = 0 Then
        Debug.Print " tokenformats: "
        For n = LBound(tokenformats) To UBound(tokenformats)
            Debug.Print " " & tokenformats(n)
        Next n
        Debug.Print " tokenvalues: "
        For n = LBound(tokenvalues) To UBound(tokenvalues)
            Debug.Print " " & tokenvalues(n)
        Next n
    End If

```

```

        Next n
    End If
    Debug.Print ""
    count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextPrefix, tokenformats, tokenvalues)
    Debug.Print "Number of prefix items: " & count
    If Not count = 0 Then
        Debug.Print " tokenformats: "
        For n = LBound(tokenformats) To UBound(tokenformats)
            Debug.Print " " & tokenformats(n)

            Next n
        Debug.Print " tokenvalues: "
        For n = LBound(tokenvalues) To UBound(tokenvalues)
            Debug.Print " " & tokenvalues(n)

            Next n
    End If
    Debug.Print ""
    count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextSuffix, tokenformats, tokenvalues)
    Debug.Print "Number of suffix items: " & count
    If Not count = 0 Then
        Debug.Print " tokenformats: "
        For n = LBound(tokenformats) To UBound(tokenformats)
            Debug.Print " " & tokenformats(n)

            Next n
        Debug.Print " tokenvalues: "
        For n = LBound(tokenvalues) To UBound(tokenvalues)
            Debug.Print " " & tokenvalues(n)

            Next n
    End If
    Debug.Print ""
    count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextCalloutBelow, tokenformats, tokenvalues)
    Debug.Print "Number of callout below items: " & count
    If Not count = 0 Then
        Debug.Print " tokenformats: "
        For n = LBound(tokenformats) To UBound(tokenformats)
            Debug.Print " " & tokenformats(n)

            Next n
        Debug.Print " tokenvalues: "
        For n = LBound(tokenvalues) To UBound(tokenvalues)
            Debug.Print "" & tokenvalues(n)

            Next n
    End If

    ' Create lower display dimension text and enclose
    ' in parentheses
    swDispDim.SetLowerText ("Example of lower text")
    swDispDim.ShowLowerParenthesis = True

    Debug.Print ("" )
    Debug.Print ("Lower Dimension Text: " & swDispDim.GetLowerText)
    Debug.Print (" Show lower parenthesis: " & swDispDim.ShowLowerParenthesis)
    Debug.Print (" Show lower inspection: " & swDispDim.LowerInspection)
    Debug.Print (" Split dual dimensions: " & swDispDim.Split)
End Sub

```

Get Chamfer Display Dimension Example (VB.NET)

This example shows how to get chamfer display dimension properties. This example also shows how to set and get lower display dimension text.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
' 2. Create a drawing from the part:
'   a. File > Make Drawing from Part.
'   b. Drag and drop the isometric view onto the sheet.
' 3. Create a chamfer display dimension.
'   a. Select Tools > Dimensions > Chamfer.
'   b. Select a chamfer edge.
'   c. Select a leading edge.
'   d. Click outside the part to place the display dimension.
'   e. Modify display dimension text in the PropertyManager:
'       1. In Dimension Text, click before <DIM>.
'       2. Select the diameter symbol.
'       3. Click after <DIM>.
'       4. Select the +/- symbol.
'       5. Type 0.5.
'       6. On the Other tab, select Override Units.
'       7. Click the green check mark to accept the
'          display dimension.
' 4. Open an Immediate Window.
' 5. Run the macro.
'
' Postconditions: Inspect the Immediate Window and the display
' dimension in the graphics area.
'
' NOTE: Because the part document is used elsewhere,
' do not save any changes when closing it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swDispDim As DisplayDimension
    Dim bRet As Boolean
    Sub main()

        swModel = swApp.ActiveDoc

        swSelMgr = swModel.SelectionManager

        bRet = swModel.Extension.SelectByID2("RD1@Drawing View1", "DIMENSION", 0.4935677398765, 0.3280800260301, 0, False, 0,
Nothing, 0)
        swDispDim = swSelMgr.GetSelectedObject6(1, 0)
        swModel.ClearSelection2(True)

        Debug.Print("Dimension type as defined in swDimensionType_e: " & swDispDim.Type2) '10=swChamferDimension
        Debug.Print("")
        Debug.Print("Uses document units? " & swDispDim.GetUseDocUnits) 'false if uses local override units

        If swDispDim.GetUseDocUnits = False Then
            Dim LenUnit As Long
            Dim AngUnit As Long
            bRet = swDispDim.GetChamferUnits(LenUnit, AngUnit)
            Debug.Print("Uses local length unit as defined in swLengthUnit_e: " & LenUnit) '0=swMM
            Debug.Print("Uses local angle unit as defined in swAngleUnit_e: " & AngUnit) '0=swDegrees, 3=swRadians
        End If
        Debug.Print("")

        Dim indx As Long
        indx = swDispDim.ChamferPrecision(0)
        Debug.Print("Precision of chamfer distance: " & indx & " decimal places")
        indx = swDispDim.ChamferPrecision(1)
        Debug.Print("Precision of chamfer angle: " & indx & " decimal places")

        Dim tokenformats As Object = Nothing
        Dim tokenvalues As Object = Nothing
        Dim n As Long

        Debug.Print("")
        Debug.Print("Text format items in " & swDispDim.GetNameForSelection & ":")
        Debug.Print("")
        Dim count As Long

        count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextCalloutAbove, tokenformats, tokenvalues)
        Debug.Print("Number of callout above items: " & count)

        If Not count = 0 Then
            Debug.Print(" tokenformats: ")
            For n = LBound(tokenformats) To UBound(tokenformats)
```

```

        Debug.Print(" " & tokenformats(n))

    Next n
    Debug.Print(" tokenvalues: ")
    For n = LBound(tokenvalues) To UBound(tokenvalues)
        Debug.Print(" " & tokenvalues(n))

    Next n
End If
Debug.Print("")
count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextPrefix, tokenformats, tokenvalues)
Debug.Print("Number of prefix items: " & count)
If Not count = 0 Then
    Debug.Print(" tokenformats: ")
    For n = LBound(tokenformats) To UBound(tokenformats)
        Debug.Print(" " & tokenformats(n))

    Next n
    Debug.Print(" tokenvalues: ")
    For n = LBound(tokenvalues) To UBound(tokenvalues)
        Debug.Print(" " & tokenvalues(n))

    Next n
End If
Debug.Print("")
count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextSuffix, tokenformats, tokenvalues)
Debug.Print("Number of suffix items: " & count)

If Not count = 0 Then
    Debug.Print(" tokenformats: ")
    For n = LBound(tokenformats) To UBound(tokenformats)
        Debug.Print(" " & tokenformats(n))

    Next n
    Debug.Print(" tokenvalues: ")
    For n = LBound(tokenvalues) To UBound(tokenvalues)
        Debug.Print(" " & tokenvalues(n))

    Next n
End If
Debug.Print("")
count = swDispDim.GetTextFormatItems(swDimensionTextParts_e.swDimensionTextCalloutBelow, tokenformats, tokenvalues)
Debug.Print("Number of callout below items: " & count)

If Not count = 0 Then
    Debug.Print(" tokenformats: ")
    For n = LBound(tokenformats) To UBound(tokenformats)
        Debug.Print(" " & tokenformats(n))

    Next n
    Debug.Print(" tokenvalues: ")
    For n = LBound(tokenvalues) To UBound(tokenvalues)
        Debug.Print(" " & tokenvalues(n))

    Next n
End If

' Create lower display dimension text and enclose
' in parentheses
swDispDim.SetLowerText("Example of lower text")
swDispDim.ShowLowerParenthesis = True

Debug.Print("")
Debug.Print("Lower Dimension Text: " & swDispDim.GetLowerText)
Debug.Print(" Show lower parenthesis: " & swDispDim.ShowLowerParenthesis)
Debug.Print(" Show lower inspection: " & swDispDim.LowerInspection)
Debug.Print(" Split dual dimensions: " & swDispDim.Split)

End Sub

Public swApp As SldWorks

End Class

```

Get Chamfer Distances Example (C#)

This example shows how to get the distances associated with the selected chamfer.

```
//-----
// Preconditions:
// 1. Open a part document with at least one chamfer feature.
// 2. Select the chamfer feature.
// 3. Open the Immediate window.
//
// Postconditions: Examine the Immediate window for the
// chamfer data.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace GetFaceEdgeCount_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            // 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            const double DegPerRad = 57.3;
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            ChamferFeatureData2 swChamfer = default(ChamferFeatureData2);
            Vertex swVertex = default(Vertex);
            object[] vEdgeArr = null;
            object vEdge = null;
            Edge swEdge = default(Edge);
            object[] vFaceArr = null;
            object vFace = null;
            Face2 swFace = default(Face2);
            object[] vLoopArr = null;
            object vLoop = null;
            Loop2 swLoop = default(Loop2);
            object vLoopEdge = null;
            object[] vLoopEdgeArr = null;
            Edge swLoopEdge = default(Edge);
            Entity swEnt = default(Entity);
            SelectData swSelData = default(SelectData);
            int i = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swChamfer = (ChamferFeatureData2)swFeat.GetDefinition();

            // Get chamfer information
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swFeat.Name);
            Debug.Print("    EdgeChamferAngle      = " + swChamfer.EdgeChamferAngle * DegPerRad + " degrees");
            Debug.Print("    EqualDistance        = " + swChamfer.EqualDistance);
            Debug.Print("    EdgeChamferDistance(0) = " + swChamfer.GetEdgeChamferDistance(0) * 1000.0 + " mm");
            Debug.Print("    EdgeChamferDistance(1) = " + swChamfer.GetEdgeChamferDistance(1) * 1000.0 + " mm");
            Debug.Print("    VertexChamferDistance(0) = " + swChamfer.GetVertexChamferDistance(0) * 1000.0 + " mm");
            Debug.Print("    VertexChamferDistance(1) = " + swChamfer.GetVertexChamferDistance(1) * 1000.0 + " mm");
            Debug.Print("    VertexChamferDistance(2) = " + swChamfer.GetVertexChamferDistance(2) * 1000.0 + " mm");
            Debug.Print("    KeepFeatures         = " + swChamfer.KeepFeatures);
            Debug.Print("    Number of chamfered faces = " + swChamfer.GetFaceCount());
            Debug.Print("    Number of chamfered edges = " + swChamfer.GetEdgeCount());
            Debug.Print("    Type                  = " + swChamfer.Type);
            // ChamferFeatureData2::Type
            // 1 = Angle-Distance
            // 2 = Distance-Distance
            // 3 = Vertex

            // Roll back to get access to geometric entities
            bRet = swChamfer.AccessSelections(swModel, null);
            Debug.Assert(bRet);

            swVertex = (Vertex)swChamfer.Vertex;

            vEdgeArr = (object[])swChamfer.Edges;
            vFaceArr = (object[])swChamfer.Faces;
            vLoopArr = (object[])swChamfer.Loops;
        }
    }
}
```

```

    if ((swVertex != null))
    {
        swModel.ClearSelection2(true);
        swEnt = (Entity)swVertex;
        bRet = swEnt.Select4(true, swSelData);
        Debug.Assert(bRet);
    }

    if ((vEdgeArr != null))
    {
        swModel.ClearSelection2(true);
        i = 0;
        bRet = false;
        foreach (object vEdge_loopVariable in vEdgeArr)
        {
            vEdge = vEdge_loopVariable;
            swEdge = (Edge)vEdge;
            swEnt = (Entity)swEdge;

            bRet = swEnt.Select4(true, swSelData);
            Debug.Assert(bRet);

            Debug.Print("    EdgeFlip(" + i + " )           = " + swChamfer.GetIsFlipped(swEdge));

            i = i + 1;
        }
    }

    if ((vFaceArr != null))
    {
        swModel.ClearSelection2(true);
        i = 0;
        bRet = false;

        foreach (object vFace_loopVariable in vFaceArr)
        {
            vFace = vFace_loopVariable;
            swFace = (Face2)vFace;
            swEnt = (Entity)swFace;

            bRet = swEnt.Select4(true, swSelData);
            Debug.Assert(bRet);

            Debug.Print("    FaceFlip(" + i + " )           = " + swChamfer.GetIsFlipped(swFace));

            i = i + 1;
        }
    }

    if ((vLoopArr != null))
    {
        swModel.ClearSelection2(true);
        i = 0;
        bRet = false;

        foreach (object vLoop_loopVariable in vLoopArr)
        {
            vLoop = vLoop_loopVariable;
            swLoop = (Loop2)vLoop;

            // Cannot select loop-through-entity interface because loop
            // is topology; instead, get edges (geometry) and select through
            // entity from edge

            vLoopEdgeArr = (object[])swLoop.GetEdges();

            foreach (object vLoopEdge_loopVariable in vLoopEdgeArr)
            {
                vLoopEdge = vLoopEdge_loopVariable;
                swLoopEdge = (Edge)vLoopEdge;
                swEnt = (Entity)swLoopEdge;

                bRet = swEnt.Select4(true, swSelData);
                Debug.Assert(bRet);
            }

            Debug.Print("    LoopFlip(" + i + " )           = " + swChamfer.GetIsFlipped(swLoop));
            i = i + 1;
        }
    }

    //Cancel changes
    swChamfer.ReleaseSelectionAccess();
}

public SldWorks swApp;

```


Get Chamfer Distances Example (VBA)

This example shows how to get the distances associated with the selected chamfer.

```
'-----
' Preconditions:
' 1. Open a part document that contains at least
'   one chamfer feature.
' 2. Select a chamfer feature.
' 3. Open the Immediate window.
'
' Postconditions: Examine the Immediate window for
' the chamfer data.
'-----
```

Option Explicit

Sub main()

```
    ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Const DegPerRad As Double = 57.3
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swChamfer As SldWorks.ChamferFeatureData2
    Dim swVertex As SldWorks.Vertex
    Dim vEdgeArr As Variant
    Dim vEdge As SldWorks.Edge
    Dim vFaceArr As Variant
    Dim vFace As SldWorks.Face2
    Dim swFace As SldWorks.Face2
    Dim vLoopArr As Variant
    Dim vLoop As SldWorks.Loop
    Dim swLoop As SldWorks.Loop2
    Dim vLoopEdge As SldWorks.Edge
    Dim vLoopEdgeArr As Variant
    Dim swLoopEdge As SldWorks.Edge
    Dim swEnt As SldWorks.Entity
    Dim swSelData As SldWorks.SelectData
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swChamfer = swFeat.GetDefinition

    ' Get chamfer information
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name
    Debug.Print " EdgeChamferAngle = " & swChamfer.EdgeChamferAngle * DegPerRad & " degrees"
    Debug.Print " EqualDistance = " & swChamfer.EqualDistance
    Debug.Print " EdgeChamferDistance(0) = " & swChamfer.GetEdgeChamferDistance(0) * 1000# & " mm"
    Debug.Print " EdgeChamferDistance(1) = " & swChamfer.GetEdgeChamferDistance(1) * 1000# & " mm"
    Debug.Print " VertexChamferDistance(0) = " & swChamfer.GetVertexChamferDistance(0) * 1000# & " mm"
    Debug.Print " VertexChamferDistance(1) = " & swChamfer.GetVertexChamferDistance(1) * 1000# & " mm"
    Debug.Print " VertexChamferDistance(2) = " & swChamfer.GetVertexChamferDistance(2) * 1000# & " mm"
    Debug.Print " KeepFeatures = " & swChamfer.KeepFeatures
    Debug.Print " Number of chamfered faces = " & swChamfer.GetFaceCount
    Debug.Print " Number of chamfered edges = " & swChamfer.GetEdgeCount
    Debug.Print " Type = " & swChamfer.Type
    ' ChamferFeatureData2::Type
    ' 1 = Angle-Distance
    ' 2 = Distance-Distance
    ' 3 = Vertex

    ' Roll back to get access to geometric entities
    bRet = swChamfer.AccessSelections(swModel, Nothing): Debug.Assert bRet

    Set swVertex = swChamfer.Vertex
    vEdgeArr = swChamfer.Edges
    vFaceArr = swChamfer.Faces
    vLoopArr = swChamfer.Loops

    If Not swVertex Is Nothing Then
        swModel.ClearSelection2 True
        Set swEnt = swVertex
        bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet
    End If

    If Not IsEmpty(vEdgeArr) Then
        swModel.ClearSelection2 True
        i = 0
        bRet = False
        For Each vEdge In vEdgeArr
            Set swEdge = vEdge
            Set swEnt = swEdge
        Next
    End If
```

```

        bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet
        Debug.Print "    EdgeFlip(" & i & ")          = " & swChamfer.GetIsFlipped(swEdge)
        i = i + 1
    Next
End If

If Not IsEmpty(vFaceArr) Then
    swModel.ClearSelection2 True
    i = 0
    bRet = False
    For Each vFace In vFaceArr
        Set swFace = vFace
        Set swEnt = swFace

        bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet

        Debug.Print "    FaceFlip(" & i & ")          = " & swChamfer.GetIsFlipped(swFace)
        i = i + 1
    Next
End If

If Not IsEmpty(vLoopArr) Then
    swModel.ClearSelection2 True
    i = 0
    bRet = False
    For Each vLoop In vLoopArr
        Set swLoop = vLoop
        ' Cannot select loop-through-entity interface because loop
        ' is topology; instead, get edges (geometry) and select through
        ' entity from edge
        vLoopEdgeArr = swLoop.GetEdges
        For Each vLoopEdge In vLoopEdgeArr
            Set swLoopEdge = vLoopEdge
            Set swEnt = swLoopEdge
            bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet
        Next
        Debug.Print "    LoopFlip(" & i & ")          = " & swChamfer.GetIsFlipped(swLoop)
        i = i + 1
    Next
End If
'Cancel changes
swChamfer.ReleaseSelectionAccess
End Sub

```

Get Chamfer Distances Example (VB.NET)

This example shows how to get the distances associated with the selected chamfer.

```

'-----
' Preconditions:
' 1. Open a part document with at least on chamfer.
' 2. Select a chamfer feature.
' 3. Open the Immediate window.
'-----
' Postconditions: Examine the Immediate window for
' the chamfer data.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()
        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Const DegPerRad As Double = 57.3
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swChamfer As ChamferFeatureData2
        Dim swVertex As Vertex
        Dim vEdgeArr As Object
        Dim vEdge As Object
        Dim swEdge As Edge
        Dim vFaceArr As Object
        Dim vFace As Object
        Dim swFace As Face2
        Dim vLoopArr As Object
        Dim vLoop As Object
        Dim swLoop As Loop2
        Dim vLoopEdge As Object
        Dim vLoopEdgeArr As Object
        Dim swLoopEdge As Edge
        Dim swEnt As Entity
        Dim swSelData As SelectData
        Dim i As Integer
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swChamfer = swFeat.GetDefinition

        ' Get chamfer information
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swFeat.Name)
        Debug.Print("    EdgeChamferAngle          = " & swChamfer.EdgeChamferAngle * DegPerRad & " degrees")
        Debug.Print("    EqualDistance          = " & swChamfer.EqualDistance)
        Debug.Print("    EdgeChamferDistance(0) = " & swChamfer.GetEdgeChamferDistance(0) * 1000.0# & " mm")
        Debug.Print("    EdgeChamferDistance(1) = " & swChamfer.GetEdgeChamferDistance(1) * 1000.0# & " mm")
        Debug.Print("    VertexChamferDistance(0) = " & swChamfer.GetVertexChamferDistance(0) * 1000.0# & " mm")
        Debug.Print("    VertexChamferDistance(1) = " & swChamfer.GetVertexChamferDistance(1) * 1000.0# & " mm")
        Debug.Print("    VertexChamferDistance(2) = " & swChamfer.GetVertexChamferDistance(2) * 1000.0# & " mm")
        Debug.Print("    KeepFeatures          = " & swChamfer.KeepFeatures)
        Debug.Print("    Number of chamfered faces = " & swChamfer.GetFaceCount)
        Debug.Print("    Number of chamfered edges = " & swChamfer.GetEdgeCount)
        Debug.Print("    Type                  = " & swChamfer.Type)
        ' ChamferFeatureData2::Type
        ' 1 = Angle-Distance
        ' 2 = Distance-Distance
        ' 3 = Vertex

        ' Roll back to get access to geometric entities
        bRet = swChamfer.AccessSelections(swModel, Nothing) : Debug.Assert(bRet)

        swVertex = swChamfer.Vertex

        vEdgeArr = swChamfer.Edges
        vFaceArr = swChamfer.Faces
        vLoopArr = swChamfer.Loops

        If Not swVertex Is Nothing Then
            swModel.ClearSelection2(True)
            swEnt = swVertex
            bRet = swEnt.Select4(True, swSelData) : Debug.Assert(bRet)
        End If

        If Not IsNothing(vEdgeArr) Then
            swModel.ClearSelection2(True)
            i = 0

```

```

bRet = False
For Each vEdge In vEdgeArr
    swEdge = vEdge
    swEnt = swEdge

    bRet = swEnt.Select4(True, swSelData) : Debug.Assert(bRet)

    Debug.Print("    EdgeFlip(" & i & ")          = " & swChamfer.GetIsFlipped(swEdge))

    i = i + 1
Next

End If

If Not IsNothing(vFaceArr) Then
    swModel.ClearSelection2(True)
    i = 0
    bRet = False

    For Each vFace In vFaceArr
        swFace = vFace
        swEnt = swFace

        bRet = swEnt.Select4(True, swSelData) : Debug.Assert(bRet)

        Debug.Print("    FaceFlip(" & i & ")          = " & swChamfer.GetIsFlipped(swFace))

        i = i + 1
    Next

End If

If Not IsNothing(vLoopArr) Then
    swModel.ClearSelection2(True)
    i = 0
    bRet = False

    For Each vLoop In vLoopArr
        swLoop = vLoop

        ' Cannot select loop-through-entity interface because loop
        ' is topology; instead, get edges (geometry) and select through
        ' entity from edge

        vLoopEdgeArr = swLoop.GetEdges

        For Each vLoopEdge In vLoopEdgeArr
            swLoopEdge = vLoopEdge
            swEnt = swLoopEdge

            bRet = swEnt.Select4(True, swSelData) : Debug.Assert(bRet)
            Next

            Debug.Print("    LoopFlip(" & i & ")          = " & swChamfer.GetIsFlipped(swLoop))
            i = i + 1
        Next

    End If

    'Cancel changes
    swChamfer.ReleaseSelectionAccess()

End Sub

Public swApp As SldWorks

End Class

```

Get Circular Holes In Face Example (C++ COM)

This example shows how to calculate the number of circular holes in a selected face. IEdge::IGetCurveParams2 and ILoop::IGetEdges return arrays, so this code must be used in an in-process DLL. Otherwise, use the methods IEdge::GetCurveParams2 and IEdge::GetEdges that return VARIANTS.

```
void GetCountHole1(LPMODELDOC2 pModelDoc, CString* Message)
{
    long lres=0;
    int pCount=0;
    LPSELECTIONMGR pSelectMgr=NULL;
    HRESULT hres=NULL;
    LPUNKNOWN pUnk=NULL;
    LPFACE SelectedFace=NULL;
    LPLOOP pLoop=NULL,pFirstLoop=NULL;
    VARIANT_BOOL pVBool,pCBool;
    LPEDGE pEdge=NULL;
    LPCURVE pCurve=NULL;
    char ch[128];
    short i;
    long pEdgeCount;

    //Form the return message
    *Message = "";

    //Get the selection mgr for this doc
    if(S_OK!=pModelDoc->get_ISelectionManager(&pSelectMgr)||pSelectMgr==NULL)
    {
        *Message+="Unable to get the Selection Manager.\r\n";
    }
    else if(S_OK!=pSelectMgr->GetSelectedObjectType(1,&lres)||lres==0)
    {
        //Get the selected object type
        *Message+="No face is currently selected.\r\n";
    }
    else if(S_OK!=pSelectMgr->IGetSelectedObject2(1,&pUnk)||pUnk==NULL)
    {
        //Get the selected object of Unknown Type
        *Message+="Unable to get the selected object.\r\n";
    }
    else if(S_OK!=pUnk->QueryInterface(IID_IFace, (LPVOID *)&SelectedFace)||SelectedFace==NULL)
    {
        //Get the selected face
        *Message+="Unable to get the selected face.\r\n";
    }
    else if(S_OK!=SelectedFace->IGetFirstLoop(&pLoop)||pLoop==NULL)
    {
        //Get the selected loop
        *Message+="Unable to get the selected face first loop.\r\n";
    }
    else
    {
        pFirstLoop=pLoop;
        do
        {
            if(S_OK!=pLoop->IsOuter(&pVBool)||pVBool==TRUE)
            {
                // Check whether the loop is outer
                *Message+="The loop is an outer loop.\r\n";
            }
            else if(S_OK!=pLoop->GetEdgeCount(&pEdgeCount)||pEdgeCount<=0)
            {
                // Check the loop has edges, for example, no vertex loops.
                *Message+="The loop has no edges.\r\n";
            }
            else
            {
                // Allocate the calculated size edge list array and set them to null
            }
        }
    }
}
```

```

LPEDGE *pEdgeList = new LPEDGE[pEdgeCount];
for (i=0;i<pEdgeCount;i++)
{
    pEdgeList[i]=NULL;
}
// Get the edge list
if(S_OK!=pLoop->IGetEdges(&pEdgeList)||pEdgeList==NULL)
{
    *Message+="Cannot get the loop edges.\r\n";
}
else
{
    // Allocate the fixed size edge data array
    double* pEdgeData = new double[100];

    // Get the first edge in the list
    pEdge=pEdgeList[0];

    if(S_OK!=pEdge->IGetCurve(&pCurve)||pCurve==NULL)
    {
        // Get the curve
        *Message+="Cannot get the edge curve.\r\n";
    }
    else if(S_OK!=pEdge->IGetCurveParams2(&pEdgeData[0]))
    {
        // Get the data of curve
        *Message+="Failed to get the curve parameters.\r\n";
    }
    else
    {
        // Get the edge curve data
        if(S_OK!=pCurve->IsCircle(&pCBool)||pCBool==FALSE)
        {
            // the curve is not a circle
            *Message+="The curve is not a circle.\r\n";
        }
        // Increment the circular hole count
        else pCount=pCount+1;
    }
    // Release the curve
    if(pCurve!=NULL)pCurve->Release();

    delete pEdgeData;

}

// Release the allocated SOLIDWORKS edge list objects
for (i=0;i<pEdgeCount;i++)
{
    if(pEdgeList[i]!=NULL)pEdgeList[i]->Release();
}
delete pEdgeList;
}

// Get the next loop
}while(S_OK==pLoop->IGetNext(&pLoop)&&pLoop!=NULL&&pLoop!=pFirstLoop);

int t=sprintf(ch,"%d",pCount);
CString pStr(ch);
*Message+="The number of circular holes is " + pStr + ".\r\n";
}

// Everything is okay.
// =====
// Release the loop
if(pLoop!=NULL)pLoop->Release();
// Release the selected face
if(SelectedFace!=NULL)SelectedFace->Release();
// Release the Selection Manager
if(pSelectMgr!=NULL)pSelectMgr->Release();
// Release the unknown object
if(pUnk!=NULL)pUnk->Release();
// Return with report of results including errors
return;

```


Get Collapsed Transform of Component in Exploded View Example (C#)

This example shows how to get the collapsed transform of a component in an exploded view.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the name of the active configuration.
// 2. Creates five exploded views for the active configuration.
// 3. Gets the name of each exploded view for the active configuration
// and shows each exploded view.
// 4. Gets the name of the exploded view shown in the model.
// 5. Selects a component and get its collapsed transform.
// 6. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Configuration swConfig = default(Configuration);
            Component2 swComponent = default(Component2);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            MathTransform swTransform = default(MathTransform);
            string activeConfigName = null;
            string[] viewNames = null;
            string viewName = null;
            int i = 0;
            double[] transformArrayData = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;

            //Get active configuration name
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfig = (Configuration)swConfigMgr.ActiveConfiguration;
            activeConfigName = swConfig.Name;

            //Create five exploded views in the active configuration
            for (i = 0; i <= 4; i++)
            {
                swAssembly.CreateExplodedView();
            }

            //Get the name of each exploded view in the active configuration
            //and show each exploded view
            viewNames = (string[])swAssembly.GetExplodedViewNames2(activeConfigName);
            for (i = 0; i < viewNames.Length; i++)
            {
                viewName = viewNames[i];
                swAssembly.ShowExploded2(true, viewName);
            }

            //Get the name of exploded view shown in model
            viewName = "";
            swModelDocExt = swModel.Extension;
            swModelDocExt.IsExploded(out viewName);
            Debug.Print("Name of exploded view shown in model: " + viewName);

            //Select a component and get its collapsed transform
            swModelDocExt.SelectByID2("speaker_frame-1@speaker", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swComponent = (Component2)swSelectionMgr.GetSelectedObjectsComponent4(1, -1);
            Debug.Print(" Name of component whose collapsed transform to get in the exploded view: " + swComponent.Name2);
            swTransform = (MathTransform)swComponent.GetSpecificTransform(true);
            transformArrayData = (double[])swTransform.ArrayData;
            Debug.Print(" Transform:");
            Debug.Print(" Rotate = (" + transformArrayData[0].ToString("###0.0#####") + ", " + transformArrayData[1].ToString("###0.0#####") + ", "
+ transformArrayData[2].ToString("###0.0#####") + ")");
            Debug.Print(" Translate = (" + transformArrayData[3].ToString("###0.0#####") + " " + transformArrayData[4].ToString("###0.0#####") + " "
+ transformArrayData[5].ToString("###0.0#####") + ")");
            Debug.Print(" Scale = (" + transformArrayData[6].ToString("###0.0#####") + " " + transformArrayData[7].ToString("###0.0#####") + " "
+ transformArrayData[8].ToString("###0.0#####") + ")");
            Debug.Print(" Translate = (" + transformArrayData[9].ToString("###0.0#####") + " " + transformArrayData[10].ToString("###0.0#####") + " "
+ transformArrayData[11].ToString("###0.0#####") + ")");
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Collapsed Transform of Component in Exploded View Example (VBA)

This example shows how to get the collapsed transform of a component in an exploded view.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the name of the active configuration.
' 2. Creates five exploded views for the active configuration.
' 3. Gets the name of each exploded view for the active configuration
'    and shows each exploded view.
' 4. Gets the name of the exploded view shown in the model.
' 5. Selects a component and gets its collapsed transform.
' 6. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim swConfigMgr As SldWorks.ConfigurationManager
Dim swConfig As SldWorks.Configuration
Dim swComponent As SldWorks.Component2
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swTransform As SldWorks.MathTransform
Dim activeConfigName As String
Dim viewNames As Variant
Dim viewName As String
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssembly = swModel

    'Get active configuration name
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    activeConfigName = swConfig.Name

    'Create five exploded views in the active configuration
    For i = 0 To 4
        swAssembly.CreateExplodedView
    Next i

    'Get the name of each exploded view in the active configuration
    'and show each exploded view
    viewNames = swAssembly.GetExplodedViewNames2(activeConfigName)
    For i = 0 To UBound(viewNames)
        viewName = viewNames(i)
        swAssembly.ShowExploded2 True, viewName
    Next i

    'Get the name of exploded view shown in model
    viewName = ""
    Set swModelDocExt = swModel.Extension
    swModelDocExt.IsExploded viewName
    Debug.Print "Name of exploded view shown in model: " & viewName

    'Select a component and get its collapsed transform
    swModelDocExt.SelectByID2 "speaker_frame-1@speaker", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0
    Set swSelectionMgr = swModel.SelectionManager
    Set swComponent = swSelectionMgr.GetSelectedObjectsComponent4(1, -1)
    Debug.Print " Name of component whose collapsed transform to get in the exploded view: " & swComponent.Name2

    Set swTransform = swComponent.GetSpecificTransform(True)
    Debug.Print " Transform:"
    Debug.Print " Rotate = (" & swTransform.ArrayData(0) & ", " & swTransform.ArrayData(1) & ", " & swTransform.ArrayData(2) & ")"
    Debug.Print " = (" & swTransform.ArrayData(3) & ", " & swTransform.ArrayData(4) & ", " & swTransform.ArrayData(5) & ")"
    Debug.Print " = (" & swTransform.ArrayData(6) & ", " & swTransform.ArrayData(7) & ", " & swTransform.ArrayData(8) & ")"
    Debug.Print " Translate = (" & swTransform.ArrayData(9) & ", " & swTransform.ArrayData(10) & ", " & swTransform.ArrayData(11) & ")"

End Sub

```

Get Collapsed Transform of Component in Exploded View Example (VB.NET)

This example shows how to get the collapsed transform of a component in an exploded view.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Gets the name of the active configuration.
' 2. Creates five exploded views for the active configuration.
' 3. Gets the name of each exploded view for the active configuration
'    and shows each exploded view.
' 4. Gets the name of the exploded view shown in the model.
' 5. Selects a component and gets its collapsed transform.
' 6. Examines the Immediate window.
'-----
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim swConfigMgr As ConfigurationManager
        Dim swConfig As Configuration
        Dim swComponent As Component2
        Dim swSelectionMgr As SelectionMgr
        Dim swTransform As MathTransform
        Dim activeConfigName As String
        Dim viewNames As Object
        Dim viewName As String
        Dim i As Integer

        swModel = swApp.ActiveDoc
        swAssembly = swModel

        'Get active configuration name
        swConfigMgr = swModel.ConfigurationManager
        swConfig = swConfigMgr.ActiveConfiguration
        activeConfigName = swConfig.Name

        'Create five exploded views in the active configuration
        For i = 0 To 4
            swAssembly.CreateExplodedView()
        Next i

        'Get the name of each exploded view in the active configuration
        'and show each exploded view
        viewNames = swAssembly.GetExplodedViewNames2(activeConfigName)
        For i = 0 To UBound(viewNames)
            viewName = viewNames(i)
            swAssembly.ShowExploded2(True, viewName)
        Next i

        'Get the name of exploded view shown in model
        viewName = ""
        swModelDocExt = swModel.Extension
        swModelDocExt.IsExploded(viewName)
        Debug.Print("Name of exploded view shown in model: " & viewName)

        'Select a component and get its collapsed transform
        swModelDocExt.SelectByID2("speaker_frame-1@speaker", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swComponent = swSelectionMgr.GetSelectedObjectsComponent4(1, -1)
        Debug.Print(" Name of component whose collapsed transform to get in the exploded view: " & swComponent.Name2)
        swTransform = swComponent.GetSpecificTransform(True)
        Debug.Print(" Transform:")
        Debug.Print(" Rotate = (" & swTransform.ArrayData(0) & ", " & swTransform.ArrayData(1) & ", " & swTransform.ArrayData(2) & ")")
        Debug.Print("          = (" & swTransform.ArrayData(3) & ", " & swTransform.ArrayData(4) & ", " & swTransform.ArrayData(5) & ")")
        Debug.Print("          = (" & swTransform.ArrayData(6) & ", " & swTransform.ArrayData(7) & ", " & swTransform.ArrayData(8) & ")")
        Debug.Print(" Translate = (" & swTransform.ArrayData(9) & ", " & swTransform.ArrayData(10) & ", " & swTransform.ArrayData(11) & ")")

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

End Class

Get COLORREF Values of Standard User-interface Elements (C#)

This example shows how to get the COLORREF values of standard SOLIDWORKS Design user-interface elements.

```
//-----
// Preconditions: Open the Immediate window.
//
// Postconditions:
// 1. Prints the names of the SOLIDWORKS Design standard
//    user-interface, elements (dimensions, backgrounds,
//    drawing paper, sketch status, annotations, etc.) and
//    their COLORREF values to the Immediate window.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace ColorTableCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ColorTable swColorTable = default(ColorTable);
            int standardCount = 0;
            int count = 0;
            string colorName = null;
            int colorRef = 0;

            swColorTable = (ColorTable)swApp.GetColorTable();

            standardCount = swColorTable.GetStandardCount();
            Debug.Print("SOLIDWORKS Design standard user-interface element and COLORREF value:");
            // Iterate through standard colors
            for (count = 0; count <= (standardCount - 1); count++)
            {
                // Get the entry name
                colorName = swColorTable.GetNameAtIndex(count);
                if (!string.IsNullOrEmpty(colorName))
                {
                    // Get the entry's COLORREF
                    colorRef = swColorTable.GetColorRefAtIndex(count);
                    Debug.Print(" " + colorName + " : " + colorRef);
                }
                else
                {
                    {
                }
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Get COLORREF Values of Standard User-interface Elements (VBA)

This example shows how to get the COLORREF values of SOLIDWORKS Design standard user-interface elements.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Prints the names of the SOLIDWORKS Design standard  
'    user-interface, elements (dimensions, backgrounds,  
'    drawing paper, sketch status, annotations, etc.) and  
'    their COLORREF values to the Immediate window.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swColorTable As SldWorks.colortable  
Dim standardCount As Long  
Dim count As Long  
Dim colorName As String  
Dim colorRef As Long  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swColorTable = swApp.GetColorTable  
  
standardCount = swColorTable.GetStandardCount  
Debug.Print "SOLIDWORKS Design standard user-interface element and COLORREF value:"  
' Iterate through standard colors  
For count = 0 To (standardCount - 1)  
    ' Get the entry name  
    colorName = swColorTable.GetNameAtIndex(count)  
    If colorName <> "" Then  
        ' Get the entry's COLORREF  
        colorRef = swColorTable.GetColorRefAtIndex(count)  
        Debug.Print " " & colorName & " : " & colorRef  
    Else  
        End If  
    Next count  
End Sub
```

Get COLORREF Values of Standard User-interface Elements (VB.NET)

This example shows how to get the COLORREF values of standard SOLIDWORKS Design user-interface elements.

```

'-----
' Preconditions: Open the Immediate window.
'
' Postconditions:
' 1. Prints the names of the SOLIDWORKS Design standard
'    user-interface, elements (dimensions, backgrounds,
'    drawing paper, sketch status, annotations, etc.) and
'    their COLORREF values to the Immediate window.
' 2. Examine the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swColorTable As ColorTable
        Dim standardCount As Integer
        Dim count As Integer
        Dim colorName As String
        Dim colorRef As Integer

        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS
2010\templates\part.prtdot", 0, 0, 0)

        swColorTable = swApp.GetColorTable

        standardCount = swColorTable.GetStandardCount

        Debug.Print("SOLIDWORKS Design standard
user-interface element and COLORREF value:")

        ' Iterate through standard colors
        For count = 0 To (standardCount - 1)
            ' Get the entry name
            colorName = swColorTable.GetNameAtIndex(count)

            If colorName <> "" Then

                ' Get the
entry's COLORREF

                colorRef = swColorTable.GetColorRefAtIndex(count)

                Debug.Print("  " & colorName & " : " & colorRef)

            Else
                End If
        Next count

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>

    Public swApp As SldWorks

```

End Class

Get ColorTable Standard Entries Example (VBA)

This example shows how to get the names of all standard colors and the corresponding COLORREF values.

```
'-----  
' Get the ColorTable Object  
Set ColorTable = Part.GetColorTable()  
  
' Iterate through standard colors  
For Count = 0 To ColorTable.GetStandardCount() - 1  
    ' Get the entry name  
    ColorName = ColorTable.GetNameAtIndex(Count)  
    ' Get the Entry COLORREF  
    ColorRef = ColorTable.GetColorRefAtIndex(Count)  
Next Count
```

Get Comments in Comments Folder Example (C#)

This example shows how to add and get the comments in a Comments folder.

```
//-----
// Preconditions:
// 1. Verify that the specified model document
//    to open exists.
// 2. Open Immediate window.
//
// Postconditions:
// 1. Examine the FeatureManager design tree
//    to verify that the Comments folder is not
//    shown, then press F5.
// 2. Adds a comment to the Comments folder
//    and updates the FeatureManager design
//    tree.
// 3. Examine the FeatureManager design tree
//    to verify that the Comments folder is
//    shown, then press F5.
// 4. Prints the number of comments, name of the
//    comments, and text of the comments
//    in the Comments folder to the Immediate
//    window. Examine the Immediate window.
//
// NOTE: Because this model document is used
// elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace UpdateFeatureTreeCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            {
                ModelDoc2 swModelDoc = default(ModelDoc2);
                Feature swFeat = default(Feature);
                FeatureManager swFeatMgr = default(FeatureManager);
                CommentFolder swCommentFolder = default(CommentFolder);
                Comment swComment = default(Comment);
                int nbrComments = 0;
                string sFeatType = null;
                object[] vComments = null;
                string fileName = null;
                int errors = 0;
                int warnings = 0;
                long i = 0;

                // Open part document
                fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.sldprt";
                swModelDoc = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

                // Traverse the FeatureManager design tree for Comments folder

                // Get first feature in FeatureManager design tree
                swFeat = (Feature)swModelDoc.FirstFeature();
                swFeatMgr = (FeatureManager)swModelDoc.FeatureManager;

                System.Diagnostics.Debugger.Break();
                // Examine FeatureManager design tree to verify that
                // the the Comments folder is not shown; press F5

                while ((swFeat != null))
                {
                    sFeatType = swFeat.GetTypeName();

                    // If Comments folder, add a comment
                    if (sFeatType == "CommentsFolder")
                    {
                        swCommentFolder = (CommentFolder)swFeat.GetSpecificFeature2();

                        // Add comment and update FeatureManager design tree
                        // so that Comments folder is shown
                        swComment = (Comment)swCommentFolder.AddComment("First comment in comment folder.");
                        swFeatMgr.UpdateFeatureTree();

                        System.Diagnostics.Debugger.Break();
                        // Locate Comments folder in FeatureManager design; press F5

                        // Get number of comments in Comment folder
                        nbrComments = swCommentFolder.GetCommentCount();
                        Debug.Print("Number of comments in Comments folder" + nbrComments);

                        vComments = (object[])swCommentFolder.GetComments();
                        for (i = 0; i <= (nbrComments - 1); i++)
                        {
                            swComment = (Comment)vComments[i];
                            Debug.Print("Name of comment in FeatureManager design tree = " + swComment.Name);
                            Debug.Print("Text of comment" + swComment.Text);
                            Debug.Print("");
                        }
                    }
                }
            }
        }
    }
}
```

```

    }

    // Get next feature in FeatureManager design tree
    swFeat = (Feature)swFeat.GetNextFeature();
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Get Comments in Comments Folder Example (VBA)

This example shows how to add and get the comments in a Comments folder.

```
'-----
' Preconditions:
' 1. Verify that the specified model document to
'    open exists.
' 2. Open Immediate window.
'
' Postconditions:
' 1. Examine the FeatureManager design tree
'    to verify that the Comments folder is not
'    shown, then press F5.
' 2. Adds a comment to the Comments folder
'    and updates the FeatureManager design
'    tree.
' 3. Examine the FeatureManager design tree
'    to verify that the Comments folder is
'    shown, then press F5.
' 4. Prints the number of comments, name of the
'    the comments, and text of the comments
'    in the Comments folder to the Immediate
'    window. Examine the Immediate window.
'
' NOTE: Because this model document is used
' elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swFeat As SldWorks.Feature
Dim swFeatMgr As SldWorks.FeatureManager
Dim swCommentFolder As SldWorks.CommentFolder
Dim swComment As SldWorks.Comment
Dim nbrComments As Long
Dim sFeatType As String
Dim vComments As Variant
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim i As Long

Sub main()

Set swApp = Application.SldWorks

' Open part document
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
Set swModelDoc = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

' Traverse the FeatureManager design tree for Comments folder

' Get first feature in FeatureManager design tree
Set swFeat = swModelDoc.FirstFeature
Set swFeatMgr = swModelDoc.FeatureManager

Stop
' Examine FeatureManager design tree to verify that
' the the Comments folder is not shown; press F5

Do While Not swFeat Is Nothing
    sFeatType = swFeat.GetTypeName

    ' If Comments folder, add a comment
    If sFeatType = "CommentsFolder" Then
        Set swCommentFolder = swFeat.GetSpecificFeature2

        ' Add comment and update FeatureManager design tree
        ' so that Comments folder is shown
        Set swComment = swCommentFolder.AddComment("First comment in comment folder.")
        swFeatMgr.UpdateFeatureTree

        Stop
        ' Locate Comments folder in FeatureManager design; press F5

        ' Get number of comments in Comment folder
        nbrComments = swCommentFolder.GetCommentCount
        Debug.Print "Number of comments in Comments folder      = " & nbrComments

        vComments = swCommentFolder.GetComments
        For i = 0 To (nbrComments - 1)
            Set swComment = vComments(i)
            Debug.Print "Name of comment in FeatureManager design tree = " & swComment.Name
            Debug.Print "Text of comment                        = " & swComment.Text
            Debug.Print ""
        Next i
    End If

    ' Get next feature in FeatureManager design tree
    Set swFeat = swFeatMgr.GetNextFeature
```

Loop

End Sub

Get Comments in Comments Folder Example (VB.NET)

This example shows how to add and get the comments in a Comments folder.

```

-----
Preconditions:
1. Verify that the specified model document
   to open exists.
2. Open Immediate window.
Postconditions:
1. Examine the FeatureManager design tree
   to verify that the Comments folder is not
   shown, then press F5.
2. Adds a comment to the Comments folder
   and updates the FeatureManager design
   tree.
3. Examine the FeatureManager design tree
   to verify that the Comments folder is
   shown, then press F5.
4. Prints the number of comments, name of the
   comments, and text of the comments
   in the Comments folder to the Immediate
   window. Examine the Immediate window.
NOTE: Because this model document is used
elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModelDoc As ModelDoc2
        Dim swFeat As Feature
        Dim swFeatMgr As FeatureManager
        Dim swCommentFolder As CommentFolder
        Dim swComment As Comment
        Dim nbrComments As Integer
        Dim sFeatType As String
        Dim vComments As Object
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Long

        ' Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\2012-sm.sldprt"
        swModelDoc = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        ' Traverse the FeatureManager design tree for Comments folder

        ' Get first feature in FeatureManager design tree
        swFeat = swModelDoc.FirstFeature
        swFeatMgr = swModelDoc.FeatureManager

        Stop

        ' Examine FeatureManager design tree to verify that
        ' the the Comments folder is not shown; press F5

        Do While Not swFeat Is Nothing
            sFeatType = swFeat.GetTypeName

            ' If Comments folder, add a comment
            If sFeatType = "CommentsFolder" Then
                swCommentFolder = swFeat.GetSpecificFeature2

                ' Add comment and update FeatureManager design tree
                ' so that Comments folder is shown
                swComment = swCommentFolder.AddComment("First comment in comment folder.")
                swFeatMgr.UpdateFeatureTree()

                Stop

                ' Locate Comments folder in FeatureManager design; press F5

                ' Get number of comments in Comment folder
                nbrComments = swCommentFolder.GetCommentCount
                Debug.Print("Number of comments in Comments folder      = " & nbrComments)

                vComments = swCommentFolder.GetComments
                For i = 0 To (nbrComments - 1)
                    swComment = vComments(i)
                    Debug.Print("Name of comment in FeatureManager design tree = " & swComment.Name)
                    Debug.Print("Text of comment                      = " & swComment.Text)
                    Debug.Print("")
                Next i
            End If

            ' Get next feature in FeatureManager design tree
            swFeat = swFeatMgr.GetNextFeature

        Loop

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Get Component by Name Example (C#)

This examples shows how to get a component by name and to determine if that component is fixed or floating.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\EDraw\claw\claw-mechanism.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the specified component.
// 2. Gets whether the specified component is fixed.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        AssemblyDoc swAssy;
        Component2 swComp;
        public void Main()
        {
            swModel = swApp.ActiveDoc as ModelDoc2;
            swAssy = swModel as AssemblyDoc;
            string compName;
            compName = "center-1";
            swComp = swAssy.GetComponentByName(compName);
            bool bRet;
            bRet = swComp.IsFixed();
            Debug.Print("Is component fixed: " + bRet);
        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Component by Name Example (VBA)

This examples shows how to get a component by name and to determine if that component is fixed or floating.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\EDraw\claw\claw-mechanism.sldasm.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the specified component.  
' 2. Gets whether the specified component is fixed.  
' 3. Examine the Immediate window.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swComp As SldWorks.Component2  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swAssy = swModel  
  
    Set swComp = swAssy.GetComponentByName("center-1")  
    Debug.Print "Is component fixed: " & swComp.IsFixed
```

End Sub

Get Component by Name Example (VB.NET)

This examples shows how to get a component by name and to determine if that component is fixed or floating.

```
-----
Preconditions:
1. Open public_documents\samples\tutorial\EDraw\claw\claw-mechanism.sldasm.
2. Open the Immediate window.
-----
Postconditions:
1. Gets the specified component.
2. Gets whether the specified component is fixed.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swAssy As AssemblyDoc
    Dim swComp As Component2

    Public Sub main()

        swModel = swApp.ActiveDoc
        swAssy = swModel
        Dim CompName As String
        CompName = "center-1"
        swComp = swAssy.GetComponentByName(CompName)
        Debug.Print("Is component fixed: " & swComp.IsFixed)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Select Component Face By Name Example (VBA)

This example shows how to find and select a face using its name.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\appearances\usb_flash_drive2.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects a component.
' 2. Selects a face on that component and names it.
' 3. Traverses the component's body and gets each face
'    on that body.
' 4. When the selected face matching the named face is found,
'    prints the selected face's name to the Immediate window.
' 5. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModel As SldWorks.ModelDoc2
Dim swComp As SldWorks.Component2
Dim swFace As SldWorks.Face2
Dim boolstatus As Long

Public Sub SelectComponentFaceByName(faceName As String)

    Dim swBody As SldWorks.Body2
    Dim swSelData As SldWorks.SelectData
    Dim currentFaceName As String

    Set swSelData = swSelMgr.CreateSelectData

    ' Get the component body
    Set swBody = swComp.GetBody()
    If (swBody Is Nothing) Then
        swApp.SendMsgToUser "Component body unavailable."
        swApp.SendMsgToUser "Make sure component is not lightweight or suppressed."
        Exit Sub
    End If

    Debug.Print "Traversing faces on component's body..."

    Set swFace = swBody.GetFirstFace
    Do While Not swFace Is Nothing
        currentFaceName = swModel.GetEntityName(swFace)
        If (currentFaceName = faceName) Then
            ' Select the face
            swFace.Select4 False, swSelData
            Debug.Print " Name of currently selected face; should match name of previously selected face: " & currentFaceName
            Exit Do
        End If
        Set swFace = swFace.GetNextFace
    Loop

    ' Subsequent code might select a second face,
    ' edge, or feature, and then mate the two
    ' items using AssemblyDoc::AddMate5

End Sub

Sub main()

    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim componentName As String
    Dim faceName As String

    Set swApp = CreateObject("SldWorks.Application")

    ' Get active assembly document
    Set swModel = swApp.ActiveDoc()
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Select a component and get its name
    boolstatus = swModelDocExt.SelectByID2("USB_connector_shell2-1@usb_flash_drive2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComp = swSelMgr.GetSelectedObject6(1, -1)
    componentName = swComp.Name2
    Debug.Print "Name of selected component: " & componentName

    swModel.ClearSelection2 True

    ' Select a face on the component and
    ' name it
    boolstatus = swModelDocExt.SelectByID2("", "FACE", -6.97433509299117E-03, 8.5197071911125E-04, 5.99999999997181E-03, False, 0, Nothing, 0)
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    boolstatus = swModel.SelectedFaceProperties(0, 0, 0, 0, 0, 0, 0, True, "SideFace")
    faceName = swModel.GetEntityName(swFace)
    Debug.Print "Name of selected face on selected component: " & faceName

    SelectComponentFaceByName faceName

End Sub

```


Get Component for Selected Entity Example (VBA)

This example shows how to get the component for the selected entity.

```
'-----  
' Preconditions:  
' 1. Open an assembly document.  
' 2. Select an entity in the graphics area.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the component.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swEnt As SldWorks.Entity  
    Dim swComp As SldWorks.Component2  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swEnt = swSelMgr.GetSelectedObject6(1, -1)  
    Set swComp = swEnt.GetComponent  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    If Not swComp Is Nothing Then  
        Debug.Print " " & swComp.Name2 & " --> " & swComp.GetPathName  
    Else  
        Debug.Print " Could not get component."  
    End If  
  
End Sub
```

Get Component from Feature Example (C#)

This example shows how to get a component from an assembly feature.

```
//-----
// Preconditions:
// 1. Open an assembly document with at least one component.
// 2. Select a feature in a component in the FeatureManager design tree.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetComponentFromFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeature = default(Feature);
            Entity swEntity = default(Entity);
            bool bValue = false;
            Component2 swComponent = default(Component2);

            // Get active document
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Check the document is an assembly
            if ((swModel.GetType() != (int)swDocumentTypes_e.swDocASSEMBLY))
            {
                return;
            }

            // Get the feature
            swFeature = (Feature)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);

            // Cast the feature to an entity
            swEntity = (Entity)swFeature;

            // Get type through entity interface
            Debug.Print("Entity type as defined in swSelectType_e: " + swEntity.GetType());
            Debug.Assert(swEntity.GetType() == (int)swSelectType_e.swSelBODYFEATURES);

            // Get type through feature interface
            // Feature inherits from Entity, so will actually call Entity::GetType
            Debug.Print("Entity type: " + swFeature.GetType());

            // Get the component for the entity
            swComponent = (Component2)swEntity.GetComponent();

            // Print component details
            Debug.Print(swComponent.Name2);
            Debug.Print(" " + swComponent.GetPathName());

            // Clear the selection lists
            swModel.ClearSelection2(true);

            // Select the feature through the Entity interface
            bValue = swEntity.Select4(false, null);

            // Print the type of the selected object
            Debug.Print("Selected object type as defined in swSelectType_e: " + ((SelectionMgr)
            (swModel.SelectionManager)).GetSelectedObjectType3(1, -1));
            Debug.Assert(((SelectionMgr)(swModel.SelectionManager)).GetSelectedObjectType3(1, -1) ==
            (int)swSelectType_e.swSelBODYFEATURES);

            // Clear the selection lists
            swModel.ClearSelection2(true);

            // Select the feature through the Feature interface
            bValue = swFeature.Select2(false, 0);

            // Print the type of the selected object
            Debug.Print("Selected object type as defined in swSelectType_e: " + ((SelectionMgr)
            (swModel.SelectionManager)).GetSelectedObjectType3(1, -1));
            Debug.Assert(((SelectionMgr)(swModel.SelectionManager)).GetSelectedObjectType3(1, -1) ==
            (int)swSelectType_e.swSelBODYFEATURES);
        }
    }
}
```

```
    public SldWorks swApp;  
}  
}
```

Get Component from Feature Example (VBA)

This example shows how to get a component from an assembly feature.

```
'-----
' Preconditions:
' 1. Open an assembly document with at least one component.
' 2. When the macro stops, select a feature in a component.
' 3. Press F5 to continue.
'
```

```
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Sub Main()
```

```
    Dim swModel      As SldWorks.ModelDoc2
    Dim swFeature     As SldWorks.Feature
    Dim swEntity      As SldWorks.Entity
    Dim bValue        As Boolean
    Dim swComponent   As SldWorks.Component2
```

```
    ' Connect to SOLIDWORKS
```

```
    Set swApp = Application.SldWorks
```

```
    ' Get active document
```

```
    Set swModel = swApp.ActiveDoc
```

```
    ' Check the document is an assembly
```

```
    If (swModel.GetType <> swDocASSEMBLY) Then
```

```
        Exit Sub
```

```
    End If
```

```
    ' Clear the selection
```

```
    swModel.ClearSelection2 True
```

```
    ' Select a feature in a part document
```

```
    Stop
```

```
    ' Get the feature
```

```
    Set swFeature = swModel.SelectionManager.GetSelectedObject6(1, -1)
```

```
    ' Cast the feature to an entity
```

```
    Set swEntity = swFeature
```

```
    ' Get type through entity interface
```

```
    Debug.Print "Entity type as defined in swSelectType_e: " & swEntity.GetType
```

```
    Debug.Assert (swEntity.GetType = swSelectType_e.swSelBODYFEATURES)
```

```
    ' Get type through feature interface
```

```
    ' Feature inherits from Entity, so will actually call Entity::GetType
```

```
    Debug.Print "Entity type as defined in swSelectType_e: " & swFeature.GetType
```

```
    Debug.Assert (swFeature.GetType = swSelectType_e.swSelBODYFEATURES)
```

```
    ' Get the component for the entity
```

```
    Set swComponent = swEntity.GetComponent
```

```
    ' Print component details
```

```
    Debug.Print swComponent.Name2
```

```
    Debug.Print " " & swComponent.GetPathName
```

```
    ' Clear the selection
```

```
    swModel.ClearSelection2 True
```

```
    ' Select the feature through the Entity interface
```

```
    bValue = swEntity.Select4(False, Nothing)
```

```
    ' Print the type of the selected object
```

```
    Debug.Print "Selected object type as defined in swSelectType_e: " & swModel.SelectionManager.GetSelectedObjectType3(1, -1)
```

```
    Debug.Assert (swModel.SelectionManager.GetSelectedObjectType3(1, -1) = swSelectType_e.swSelBODYFEATURES)
```

```
    ' Clear the selection
```

```
    swModel.ClearSelection2 True
```

```
    ' Select the feature through the Feature interface
```

```
    bValue = swFeature.Select2(False, 0)
```

```
    ' Print the type of the selected object
```

```
    Debug.Print "Selected object type as defined in swSelectType_e: " & swModel.SelectionManager.GetSelectedObjectType3(1, -1)
```

```
    Debug.Assert (swModel.SelectionManager.GetSelectedObjectType3(1, -1) = swSelectType_e.swSelBODYFEATURES)
```

```
End Sub
```

Get Component from Feature Example (VB.NET)

This example shows how to get a component from an assembly feature.

```

'-----
' Preconditions:
' 1. Open an assembly document with at least one component.
' 2. Select a feature in a component in the FeatureManager design tree.
'
' Postconditions: Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub Main()
```

```

        Dim swModel As ModelDoc2
        Dim swFeature As Feature
        Dim swEntity As Entity
        Dim bValue As Boolean
        Dim swComponent As Component2

        ' Get active document
        swModel = swApp.ActiveDoc

        ' Check the document is an assembly
        If (swModel.GetType <> swDocumentTypes_e.swDocASSEMBLY) Then
            Exit Sub
        End If

        ' Get the feature
        swFeature = swModel.SelectionManager.GetSelectedObject6(1, -1)

        ' Cast the feature to an entity
        swEntity = swFeature

        ' Get type through entity interface
        Debug.Print("Entity type as defined in swSelectType_e: " & swEntity.GetType)
        Debug.Assert(swEntity.GetType = swSelectType_e.swSelBODYFEATURES)

        ' Get type through feature interface
        ' Feature inherits from Entity, this will actually call Entity::GetType
        Debug.Print("Entity type as defined in swSelectType_e: " & swFeature.GetType)
        Debug.Assert(swFeature.GetType = swSelectType_e.swSelBODYFEATURES)

        ' Get the component for the entity
        swComponent = swEntity.GetComponent

        ' Print component details
        Debug.Print(swComponent.Name2)
        Debug.Print(" " & swComponent.GetPathName)

        ' Clear the selection lists
        swModel.ClearSelection2(True)

        ' Select the feature through the Entity interface
        bValue = swEntity.Select4(False, Nothing)

        ' Print the type of the selected object
        Debug.Print("Selected object type as defined in swSelectType_e: " & swModel.SelectionManager.GetSelectedObjectType3(1, -1))
        Debug.Assert(swModel.SelectionManager.GetSelectedObjectType3(1, -1) = swSelectType_e.swSelBODYFEATURES)

        ' Clear the selection lists
        swModel.ClearSelection2(True)

        ' Select the feature through the Feature interface
        bValue = swFeature.Select2(False, 0)

        ' Print the type of the selected object
        Debug.Print("Selected object type as defined in swSelectType_e: " & swModel.SelectionManager.GetSelectedObjectType3(1, -1))
        Debug.Assert(swModel.SelectionManager.GetSelectedObjectType3(1, -1) = swSelectType_e.swSelBODYFEATURES)

```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Get Component IDs Example (C#)

This example shows how to get the component IDs of the components in an assembly and subassemblies.

```
//-----
// Preconditions:
// 1. Open an assembly document containing nested subassemblies.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the assembly and subassemblies.
// 2. Gets the name and component ID of each component in the assembly and
//    subassemblies.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ConfigurationManager swConfMgr = default(ConfigurationManager);
            Configuration swConf = default(Configuration);
            Component2 swRootComp = default(Component2);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swConfMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConf = (Configuration)swConfMgr.ActiveConfiguration;
            swRootComp = (Component2)swConf.GetRootComponent3(true);
            Debug.Print("File = " + swModel.GetPathName());
            if (swModel.GetType() == (int)swDocumentTypes_e.swDocASSEMBLY)
            {
                TraverseComponent(swRootComp, 1);
            }
        }

        public void TraverseComponent(Component2 swComp, int nLevel)
        {
            object[] vChildComp = null;
            Component2 swChildComp = default(Component2);
            string sPadStr = "";
            int i = 0;

            for (i = 0; i <= nLevel - 1; i++)
            {
                sPadStr = sPadStr + "  ";
            }
            vChildComp = (object[])swComp.GetChildren();
            for (i = 0; i <= vChildComp.Length - 1; i++)
            {
                swChildComp = (Component2)vChildComp[i];
                Debug.Print(sPadStr + "Component name: " + swChildComp.Name2 + ", Component ID: " + swChildComp.GetID());
                TraverseComponent(swChildComp, nLevel + 1);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Component IDs Example (VBA)

This example shows how to get the component IDs of the components in an assembly and subassemblies.

```

'-----
' Preconditions:
' 1. Open an assembly document containing nested subassemblies.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the assembly and subassemblies.
' 2. Gets the name and component ID of each component in the assembly and
'    subassemblies.
' 3. Examine the Immediate window.
'-----
Option Explicit

Sub TraverseComponent(swComp As SldWorks.Component2, nLevel As Long)
    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim sPadStr As String
    Dim i As Long

    For i = 0 To nLevel - 1
        sPadStr = sPadStr + "  "
    Next i
    vChildComp = swComp.GetChildren
    For i = 0 To UBound(vChildComp)
        Set swChildComp = vChildComp(i)
        Debug.Print sPadStr & "Component name: " & swChildComp.Name2 & ", Component ID: " & swChildComp.GetID
        TraverseComponent swChildComp, nLevel + 1
    Next i
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swConfMgr = swModel.ConfigurationManager
    Set swConf = swConfMgr.ActiveConfiguration
    Set swRootComp = swConf.GetRootComponent3(True)
    Debug.Print "File = " & swModel.GetPathName
    If swModel.GetType = swDocumentTypes_e.swDocASSEMBLY Then
        TraverseComponent swRootComp, 1
    End If
End Sub

```

Get Component IDs Example (VB.NET)

This example shows how to get the component IDs of the components in an assembly and subassemblies.

```

-----
Preconditions:
1. Open an assembly document containing nested subassemblies.
2. Open the Immediate window.
Postconditions:
1. Traverses the assembly and subassemblies.
2. Gets the name and component ID of each component in the assembly and
subassemblies.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfMgr As ConfigurationManager
        Dim swConf As Configuration
        Dim swRootComp As Component2

        swModel = swApp.ActiveDoc
        swConfMgr = swModel.ConfigurationManager
        swConf = swConfMgr.ActiveConfiguration
        swRootComp = swConf.GetRootComponent3(True)
        Debug.Print("File = " & swModel.GetPathName)
        If swModel.GetType = swDocumentTypes_e.swDocASSEMBLY Then
            TraverseComponent(swRootComp, 1)
        End If

    End Sub

    Sub TraverseComponent(ByVal swComp As Component2, ByVal nLevel As Integer)
        Dim vChildComp As Object
        Dim swChildComp As Component2
        Dim sPadStr As String = ""
        Dim i As Integer

        For i = 0 To nLevel - 1
            sPadStr = sPadStr + " "
        Next i
        vChildComp = swComp.GetChildren
        For i = 0 To UBound(vChildComp)
            swChildComp = vChildComp(i)
            Debug.Print(sPadStr & "Component name: " & swChildComp.Name2 & ", Component ID: " & swChildComp.GetID)
            TraverseComponent(swChildComp, nLevel + 1)
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Component Name From Selected Entity Example (C#)

This example shows how to get the name of a component to possibly use with a future call to [IModelDocExtension::SelectByID2](#), when selectively opening the assembly document and specific components using [ISldWorks::OpenDoc7](#) and [IDocumentSpecification](#), etc. This example also shows how to get and set a component reference.

```
//-----
// Preconditions:
// 1. Open an assembly document.
// 2. Select an entity (face, edge, vertex, or loop) on any
//    component in the graphics area.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Adds a component reference to the component to which the
//    entity belongs.
// 2. Examine the Immediate window.
// 3. Locate the component to which the component reference was added
//    in the FeatureManager design tree. If necessary, use the scrollbar
//    at the bottom of the FeatureManager design tree to see the component
//    reference.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetSelectByIDStringComponent_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Entity swEntity = default(Entity);
            Component2 swComponent = default(Component2);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Get the selected entity (i.e., face, edge, vertex, or loop)
            // and get the name of its component
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swEntity = (Entity)swSelectionMgr.GetSelectedObject6(1, -1);
            swComponent = (Component2)swEntity.GetComponent();

            // Print the name of the component to which the
            // the selected entity belongs
            Debug.Print("Name of component to which the selected entity belongs: " + swComponent.GetSelectByIDString());

            // Set a component reference to this component
            swComponent.ComponentReference = "TestComponentReference";
            Debug.Print("Component reference added to the component to which the selected entity belongs: " + swComponent.ComponentReference);

            // Rebuild the assembly to see the component reference
            // beside the name of the component in the FeatureManager
            // design tree
            swModel.ForceRebuild3(true);
        }
    }
    public SldWorks swApp;
}
}
```

Get Component Name From Selected Entity Example (VBA)

This example shows how to get the name of a component to possibly use with a future call to [IModelDocExtension::SelectByID2](#), when selectively opening the assembly document and specific components using [ISldWorks::OpenDoc7](#) and [IDocumentSpecification](#), etc. This example also shows how to set and get a component reference.

```
'-----
' Preconditions:
' 1. Open an assembly document.
' 2. Select an entity (face, edge, vertex, or loop) on any
'    component in the graphics area.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Adds a component reference to the component to which the
'    entity belongs.
' 2. Examine the Immediate window.
' 3. Locate the component to which the component reference was added
'    in the FeatureManager design tree. If necessary, use the scrollbar
'    at the bottom of the FeatureManager design tree to see the component
'    reference.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swEntity As SldWorks.Entity
Dim swComponent As SldWorks.Component2

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Get the selected entity (i.e., face, edge, vertex, or loop)
    ' and get the name of its component
    Set swSelectionMgr = swModel.SelectionManager
    Set swEntity = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swComponent = swEntity.GetComponent

    ' Print the name of the component to which the
    ' the selected entity belongs
    Debug.Print "Name of component to which the selected entity belongs: " & swComponent.GetSelectByIDString

    ' Set a component reference to this component
    swComponent.ComponentReference = "TestComponentReference"
    Debug.Print "Component reference added to the component to which the selected entity belongs: " & swComponent.ComponentReference

    ' Rebuild the assembly to see the component reference
    ' beside the name of the component in the FeatureManager design
    ' tree
    swModel.ForceRebuild3 True

End Sub
```

Get Component Name From Selected Entity Example (VB.NET)

This example shows how to get the name of a component to possibly use with a future call to [IModelDocExtension::SelectByID2](#), when selectively opening the assembly document and specific components using [ISldWorks::OpenDoc7](#) and [IDocumentSpecification](#), etc. This example also shows how to get and set a component reference.

```
-----
Preconditions:
1. Open an assembly document.
2. Select an entity (face, edge, vertex, or loop) on any
   component in the graphics area.
3. Open the Immediate window.
Postconditions:
1. Adds a component reference to the component to which the
   entity belongs.
2. Examine the Immediate window.
3. Locate the component to which the component reference was added
   in the FeatureManager design tree. If necessary, use the scrollbar
   at the bottom of the FeatureManager design tree to see the component
   reference.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelectionMgr As SelectionMgr
        Dim swEntity As Entity
        Dim swComponent As Component2

        swModel = swApp.ActiveDoc

        ' Get the selected entity (i.e., face, edge, vertex, or loop)
        ' and get the name of its component
        swSelectionMgr = swModel.SelectionManager
        swEntity = swSelectionMgr.GetSelectedObject6(1, -1)
        swComponent = swEntity.GetComponent

        ' Print the name of the component to which the
        ' the selected entity belongs
        Debug.Print("Name of component to which the selected entity belongs: " & swComponent.GetSelectByIdString)

        ' Set a component reference to this component
        swComponent.ComponentReference = "TestComponentReference"
        Debug.Print("Component reference added to the component to which the selected entity belongs: " & swComponent.ComponentReference)

        ' Rebuild the assembly to see the component reference
        ' beside the name of the component to which the selected
        ' entity belongs, in the FeatureManager design tree
        swModel.ForceRebuild3(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Component Names and Types for Inplace Mate Example (VBA)

This example shows how to get the names of the components for an Inplace mate.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\bladed shaft.sldasm.
' 2. In the FeatureManager design tree, expand blade extension >
'    Mates in bladed shaft and click Inplace5.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the Inplace mate.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub ProcessMateInPlace(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swMateInPlace As SldWorks.MateInPlace)

    Dim swComp As SldWorks.Component2
    Dim nNumMateEnt As Long
    Dim i As Long

    Set swComp = swMateInPlace.Component: Debug.Assert Not Nothing Is swComp

    nNumMateEnt = swMateInPlace.GetMateEntityCount

    Debug.Print "    Component = " & swComp.Name2
    For i = 0 To nNumMateEnt - 1
        Debug.Print "        Mate Component = " & swMateInPlace.MateComponentName(i)
        Debug.Print "        Type of Inplace mate entity = " & swMateInPlace.MateEntityType(i)
    Next i

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swMateInPlace As SldWorks.MateInPlace
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swMateInPlace = swFeat.GetSpecificFeature2

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"

    ProcessMateInPlace swApp, swModel, swMateInPlace

End Sub
```

Get Component State Example (C#)

This example shows how to find out if the selected component is resolved or suppressed, hidden or visible, and whether or not it's a rigid or flexible subassembly. This example also gets the persistent ID of the selected component.

```
//-----
// Preconditions:
// 1. Ensure that the specified assembly document
//    to open exists.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Opens the assembly document.
// 2. Selects the subassembly.
// 3. Prints to the Immediate window:
//    * Paths to the assembly and subassembly documents
//    * Whether the component is hidden, fixed,
//    or suppressed
//    * Component's persistent ID
//    * Component's solving state
// 4. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetComponentStateCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssy = default(AssemblyDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swComp = default(Component2);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            // Open assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\98food_processor.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Select subassembly
            status = swModelDocExt.SelectByID2("blade shaft-1@98food processor", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swAssy = (AssemblyDoc)swModel;
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent3(1, 0);

            // Print to the Immediate window the path and state of the
            // selected component
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print("Component = " + swComp.Name2);
            Debug.Print("Path = " + swComp.GetPathName());
            Debug.Print("IsHidden = " + swComp.IsHidden(true));
            Debug.Print("IsFixed = " + swComp.IsFixed());
            Debug.Print("GetSuppression = " + swComp.GetSuppression());
            Debug.Print("ID = " + swComp.GetID());
            // 0 = if subassembly is rigid
            // 1 = if subassembly is flexible
            // -1 = selected component is a part component
            Debug.Print("Solving = " + swComp.Solving);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Get Component State Example (VBA)

This example shows how to find out if the selected component is resolved or suppressed, hidden or visible, and whether or not it's a rigid or flexible subassembly. This example also gets the persistent ID of the selected component.

```
'-----
' Preconditions:
' 1. Ensure that the specified assembly document
'    to open exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens the assembly document.
' 2. Selects the subassembly.
' 3. Prints to the Immediate window:
'    * Paths to the assembly and subassembly documents
'    * Whether the component is hidden, fixed,
'      or suppressed
'    * Component's persistent ID
'    * Component's solving state
' 4. Examine the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swModelDocExt        As SldWorks.ModelDocExtension
Dim swAssy              As SldWorks.AssemblyDoc
Dim swSelMgr             As SldWorks.SelectionMgr
Dim swComp              As SldWorks.Component2
Dim fileName            As String
Dim status              As Boolean
Dim errors              As Long
Dim warnings            As Long
```

```
Set swApp = Application.SldWorks
```

```
' Open assembly document
```

```
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food processor.sldasm"
Set swModel = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension
```

```
' Select subassembly
```

```
status = swModelDocExt.SelectByID2("blade shaft-1@98food processor", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
Set swSelMgr = swModel.SelectionManager
Set swAssy = swModel
Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)
```

```
' Print to the Immediate window the path and state of the
```

```
' selected component
Debug.Print "File = " & swModel.GetPathName
Debug.Print "  Component  = " & swComp.Name2
Debug.Print "    Path    = " & swComp.GetPathName
Debug.Print "  IsHidden   = " & swComp.IsHidden(True)
Debug.Print "  IsFixed    = " & swComp.IsFixed
Debug.Print "  GetSuppression = " & swComp.GetSuppression
Debug.Print "    ID        = " & swComp.GetID
' 0 = if subassembly is rigid
' 1 = if subassembly is flexible
' -1 = selected component is a part component
Debug.Print "    Solving    = " & swComp.Solving
```

End Sub

Get Component State Example (VB.NET)

This example shows how to find out if the selected component is resolved or suppressed, hidden or visible, and whether or not it's a rigid or flexible subassembly. This example also gets the persistent ID of the selected component.

```
'-----
' Preconditions:
' 1. Ensure that the specified assembly document
'    to open exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens the assembly document.
' 2. Selects the subassembly.
' 3. Prints to the Immediate window:
'    * Paths to the assembly and subassembly documents
'    * Whether the component is hidden, fixed,
'    or suppressed
'    * Component's persistent ID
'    * Component's solving state
' 4. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssy As AssemblyDoc
        Dim swSelMgr As SelectionMgr
        Dim swComp As Component2
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        ' Open assembly document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\98food_processor.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        ' Select subassembly
        status = swModelDocExt.SelectByID2("blade shaft-1@98food processor", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swAssy = swModel
        swComp = swSelMgr.GetSelectedObjectsComponent3(1, 0)

        ' Print to the Immediate window the path and state of the
        ' selected component
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Component = " & swComp.Name2)
        Debug.Print(" Path = " & swComp.GetPathName)
        Debug.Print(" IsHidden = " & swComp.IsHidden(True))
        Debug.Print(" IsFixed = " & swComp.IsFixed)
        Debug.Print(" GetSuppression = " & swComp.GetSuppression)
        Debug.Print(" ID = " & swComp.GetID)

        ' 0 = if subassembly is rigid
        ' 1 = if subassembly is flexible
        ' -1 = selected component is a part component
        Debug.Print(" Solving = " & swComp.Solving)
    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
    Public swApp As SldWorks

End Class
```

Get Component Via Display Dimension Example (VBA)

This example shows how to get the component associated with the selected display dimension.

```
'-----  
'  
' Preconditions:  
'      (1) Drawing is open with dimensions inserted  
'      (2) A dimension is selected  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swDispDim As SldWorks.DisplayDimension  
Dim swDim As SldWorks.Dimension  
Dim swFeat As SldWorks.feature  
Dim swEnt As SldWorks.entity  
Dim swComp As SldWorks.Component2  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swDispDim = swSelMgr.GetSelectedObject5(1)  
' Get the selected dimension  
Set swDim = swDispDim.GetDimension  
Debug.Print "Name of dimension: "; swDim.Name  
' Get the feature that owns the selected dimension  
Set swFeat = swDim.GetFeatureOwner  
Debug.Print "Name of feature: "; swFeat.Name  
' Feature is an entity  
Set swEnt = swFeat  
' Get the component for the feature whose dimension was selected  
Set swComp = swEnt.GetComponent  
Debug.Print "Name of component: "; swComp.Name2  
  
End Sub
```

Get Components Hidden In Drawing View Example (VBA)

This example shows how to get the components hidden in the selected drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing of an assembly.  
' 2. Hide some components in a drawing view (expand a drawing view,  
'    right-click the component, and click Show/Hide > Hide Component).  
' 3. Select the drawing view where you hid the components.  
'  
' Postconditions:  
' 1. Gets the hidden components in the selected drawing view.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit
```

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swDrView As SldWorks.View  
    Dim vCompArr As Variant  
    Dim vComp As Variant  
    Dim swComp As SldWorks.Component2  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swDrView = swSelMgr.GetSelectedObject6(1, -1)  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " " & swDrView.Name  
  
    vCompArr = swDrView.GetHiddenComponents  
    If IsEmpty(vCompArr) Then Exit Sub  
    For Each vComp In vCompArr  
        Set swComp = vComp  
        Debug.Print " " & swComp.Name2  
    Next  
  
End Sub
```

Get Components in Drawing View (C#)

This example shows how to get the components in a drawing view and how to change their line font styles.

```
//-----
// Preconditions:
// 1. Drawing document opened by the macro exists.
// 2. Drawing view is selected.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Specified drawing document is opened.
// 2. Drawing View1 is selected.
// 3. Gets the root and children components for Drawing
//    View1.
// 4. For each component:
//    a. Prints whether a drawing component is selected,
//       the name of the component, and the name of the
//       configuration to the Immediate window.
//    b. Disables the use of the document defaults for the
//       the component's line font style.
//    c. Sets new line style and line thickness for the component's
//       visible edges and prints the new settings and values to
//       the Immediate window.
//    d. Prints the file name of the component to the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace LineFontsDrawingComponentCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            View swView = default(View);
            DrawingComponent swRootDrawComp = default(DrawingComponent);
            object[] vDrawChildCompArr = null;
            DrawingComponent swDrawComp = default(DrawingComponent);
            Component2 swComp = default(Component2);
            ModelDoc2 swCompModel = default(ModelDoc2);
            string assemblyDrawing = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            int lineWeight = 0;
            double lineThickness = 0;

            assemblyDrawing = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\driveworksxpess\\mobile gantry.slddrw";
            swModel = (ModelDoc2)swApp.OpenDoc6(assemblyDrawing, (int)swDocumentTypes_e.swDocDRAWING,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            swDraw = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();

            status = swDraw.ActivateView("Drawing View4");
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.104008832128, 0.1163870710783, 0, false, 0, null,
0);

            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            swModel.ViewZoomtofit2();
            swRootDrawComp = (DrawingComponent)swView.RootDrawingComponent;

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" View = " + swView.Name);

            vDrawChildCompArr = (object[])swRootDrawComp.GetChildren();
            foreach (object vDrawChildComp in vDrawChildCompArr)
            {
                swDrawComp = (DrawingComponent)vDrawChildComp;

                // Drawing component selected?
                Debug.Print(" Drawing component selected = " + swDrawComp.Select(true, null));

                // Returns NULL if underlying model is open in a different configuration
                swComp = (Component2)swDrawComp.Component;

                if ((null != swComp))
                {
                    // Returns NULL if drawing is lightweight

```

```

swCompModel = (ModelDoc2)swComp.GetModelDoc2();

Debug.Print("      Component                = " + swComp.Name2);
Debug.Print("      Configuration            = " + swComp.ReferencedConfiguration);

// Turn off using document default settings for component's line font style
swDrawComp.UseDocumentDefaults = false;
Debug.Print("      Default component line font in use = " + swDrawComp.UseDocumentDefaults);
// Set new line style for visible edges
swDrawComp.SetLineStyle((int)swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible,
(int)swLineStyles_e.swLineCHAIN);
Debug.Print("      Line style for visible edges          = " +
swDrawComp.GetLineStyle((int)swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible));
// Set new line thickness for visible edges
swDrawComp.SetLineThickness((int)swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible,
(int)swLineWeights_e.swLW_CUSTOM, 0.0003);
lineWeight =
swDrawComp.GetLineThickness((int)swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible, out lineThickness);
Debug.Print("      Line weight style and thickness for visible edges = " + lineWeight + ", " + lineThickness
* 1000 + " mm");

if ((null != swCompModel))
{
    Debug.Print("      File                                = " + swCompModel.GetPathName());
    Debug.Print(" ");
}
}

}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Get Components in Drawing View Example (VBA)

This example shows how to get the components in a drawing view and how to change their line font styles.

```
'-----
' Preconditions:
' 1. Drawing document opened by the macro exists.
' 2. Drawing view is selected.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Specified drawing document is opened.
' 2. Drawing View1 is selected.
' 3. Gets the root and children components for Drawing
'    View1.
' 4. For each component:
'    a. Prints whether a drawing component is selected, the
'       name of the component, and the name of the configuration
'       to the Immediate window.
'    b. Disables the use of the document defaults for the
'       the component's line font style.
'    c. Sets new line style and line thickness for the component's
'       visible edges and prints the new settings and values to
'       the Immediate window.
'    d. Prints the file name of the component to the Immediate window.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSelData As SldWorks.SelectData
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swView As SldWorks.View
Dim swRootDrawComp As SldWorks.DrawingComponent
Dim vDrawChildCompArr As Variant
Dim vDrawChildComp As Variant
Dim swDrawComp As SldWorks.DrawingComponent
Dim swComp As SldWorks.Component2
Dim swCompModel As SldWorks.ModelDoc2
Dim assemblyDrawing As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim lineWeight As Long
Dim lineThickness As Double
```

Set swApp = Application.SldWorks

```
assemblyDrawing = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\driveworksxpess\mobile gantry.slddrw"
Set swModel = swApp.OpenDoc6(assemblyDrawing, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
```

```
Set swDraw = swModel
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager
Set swSelData = swSelMgr.CreateSelectData
```

```
status = swDraw.ActivateView("Drawing View4")
status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.104008832128, 0.1163870710783, 0, False, 0, Nothing, 0)
Set swView = swSelMgr.GetSelectedObject6(1, -1)
swModel.ViewZoomToFit2
Set swRootDrawComp = swView.RootDrawingComponent
```

```
Debug.Print "File = " & swModel.GetPathName
Debug.Print " View = " & swView.Name
```

vDrawChildCompArr = swRootDrawComp.GetChildren

```
For Each vDrawChildComp In vDrawChildCompArr
    Set swDrawComp = vDrawChildComp
```

```
    ' Drawing component selected?
    Debug.Print " Drawing component selected = " & swDrawComp.Select(True, Nothing)
```

```
    ' Returns NULL if underlying model is open in a different configuration
    Set swComp = swDrawComp.Component
```

```
    If Not Nothing Is swComp Then
        ' Returns NULL if drawing is lightweight
        Set swCompModel = swComp.GetModelDoc2
```

```
        Debug.Print " Component = " & swComp.Name2
        Debug.Print " Configuration = " & swComp.ReferencedConfiguration
```

```
        ' Turn off using document default settings for component's line font style
```

```
        swDrawComp.UseDocumentDefaults = False
```

```
        Debug.Print " Default component line font in use = " & swDrawComp.UseDocumentDefaults
```

```
        ' Set new line style for visible edges
```

```
        swDrawComp.SetLineStyle swDrawingComponentLineFontVisible, swLineCHAIN
```

```
        Debug.Print " Line style for visible edges = " & swDrawComp.GetLineStyle(swDrawingComponentLineFontVisible)
```

```
        ' Set new line thickness for visible edges
```

```

swDrawComp.SetLineThickness swDrawingComponentLineFontVisible, swLW_CUSTOM, 0.0003
lineWeight = swDrawComp.GetLineThickness(swDrawingComponentLineFontVisible, lineThickness)
Debug.Print "          Line weight style and thickness for visible edges = " & lineWeight & ", " & lineThickness * 1000 & " mm"

If Not Nothing Is swCompModel Then
    Debug.Print "          File                                = " & swCompModel.GetPathName
    Debug.Print " "
End If

End If

Next

End Sub

```

Get Components in Drawing View (VB.NET)

This example shows how to get the components in a drawing view and how to change their line font styles.

```

'-----
' Preconditions:
' 1. Drawing document opened by the macro exists.
' 2. Drawing view is selected.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Specified drawing document is opened.
' 2. Drawing View1 is selected.
' 3. Gets the root and children components for Drawing
'    View1.
' 4. For each component:
'    a. Prints whether a drawing component is selected, the
'       name of the component, and the name of the configuration
'       to the Immediate window.
'    b. Disables the use of the document defaults for the
'       the component's line font style.
'    c. Sets new line style and line thickness for the component's
'       visible edges and prints the new settings and values to
'       the Immediate window.
'    d. Prints the file name of the component to the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim swModelDocExt As ModelDocExtension
        Dim swView As View
        Dim swRootDrawComp As DrawingComponent
        Dim vDrawChildCompArr As Object
        Dim vDrawChildComp As Object
        Dim swDrawComp As DrawingComponent
        Dim swComp As Component2
        Dim swCompModel As ModelDoc2
        Dim assemblyDrawing As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim lineWeight As Integer
        Dim lineThickness As Double

        assemblyDrawing = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\driveworksxpess\mobile
gantry.slddrw"
        swModel = swApp.OpenDoc6(assemblyDrawing, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)

        swDraw = swModel
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData

        status = swDraw.ActivateView("Drawing View4")
        status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.104008832128, 0.1163870710783, 0, False, 0, Nothing,
0)

        swView = swSelMgr.GetSelectedObject6(1, -1)
        swModel.ViewZoomtofit2()
        swRootDrawComp = swView.RootDrawingComponent

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" View = " & swView.Name)

        vDrawChildCompArr = swRootDrawComp.GetChildren
        For Each vDrawChildComp In vDrawChildCompArr
            swDrawComp = vDrawChildComp

            ' Drawing component selected?
            Debug.Print(" Drawing component selected = " & swDrawComp.Select(True, Nothing))

            ' Returns NULL if underlying model is open in a different configuration
            swComp = swDrawComp.Component

            If Not Nothing Is swComp Then
                ' Returns NULL if drawing is lightweight
                swCompModel = swComp.GetModelDoc2

                Debug.Print(" Component = " & swComp.Name2)
            End If
        Next
    End Sub
End Class

```

```

        Debug.Print("          Configuration                      = " & swComp.ReferencedConfiguration)

        ' Turn off using document default settings for component's line font style
        swDrawComp.UseDocumentDefaults = False
        Debug.Print("          Default component line font in use    = " & swDrawComp.UseDocumentDefaults)
        ' Set new line style for visible edges
        swDrawComp.SetLineStyle(swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible,
swLineStyles_e.swLineCHAIN)
        Debug.Print("          Line style for visible edges              = " &
swDrawComp.GetLineStyle(swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible))
        ' Set new line thickness for visible edges
        swDrawComp.SetLineThickness(swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible,
swLineWeights_e.swLW_CUSTOM, 0.0003)
        lineWeight = swDrawComp.GetLineThickness(swDrawingComponentLineFontOption_e.swDrawingComponentLineFontVisible,
lineThickness)
        Debug.Print("          Line weight style and thickness for visible edges = " & lineWeight & ", " & lineThickness *
1000 & " mm")

        If Not Nothing Is swCompModel Then
            Debug.Print("          File                      = " & swCompModel.GetPathName)
            Debug.Print(" ")
        End If

    End If

Next

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Components in Each BOM Table Row Example (C#)

This example shows how to get the components in each row of a BOM table annotation.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
// 2. Make a drawing from the assembly.
// 3. Click Insert > Tables > Bill of Materials.
// 4. Ensure that Parts only in Bom Type is selected.
// 5. Ensure that Display configurations of the same part separate items
//    in Part Configuration Grouping is selected.
// 6. Click OK.
// 7. Click anywhere in the drawing to insert the BOM table.
//
// Postconditions:
// 1. Gets the Bill of Materials1 feature.
// 2. Gets the Default configuration.
// 3. Processes the BOM table for the Default configuration.
// 4. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void ProcessTableAnn(SldWorks swApp, ModelDoc2 swModel, TableAnnotation swTableAnn, string ConfigName)
        {
            int nNumRow = 0;
            int J = 0;
            int I = 0;
            string ItemNumber = null;
            string PartNumber = null;
            bool RowLocked;
            double RowHeight;

            Debug.Print("    Table Title: " + swTableAnn.Title);

            nNumRow = swTableAnn.RowCount;

            BomTableAnnotation swBOMTableAnn = default(BomTableAnnotation);
            swBOMTableAnn = (BomTableAnnotation)swTableAnn;

            for (J = 0; J <= nNumRow - 1; J++)
            {
                RowLocked = swTableAnn.GetLockRowHeight(J);
                RowHeight = swTableAnn.GetRowHeight(J);
                Debug.Print("    Row Number " + J + " (height = " + RowHeight + "; height locked = " + RowLocked + ")");
                Debug.Print("    Component Count: " + swBOMTableAnn.GetComponentsCount2(J, ConfigName, out ItemNumber, out PartNumt));
                Debug.Print("    Item Number: " + ItemNumber);
                Debug.Print("    Part Number: " + PartNumber);

                object[] vPtArr = null;
                Component2 swComp = null;
                object pt = null;

                vPtArr = (object[])swBOMTableAnn.GetComponents2(J, ConfigName);

                if ((vPtArr != null))
                {
                    for (I = 0; I <= vPtArr.GetUpperBound(0); I++)
                    {
                        pt = vPtArr[I];
                        swComp = (Component2)pt;
                        if ((swComp != null))
                        {
                            Debug.Print("        Component Name: " + swComp.Name2);
                            Debug.Print("        Configuration Name: " + swComp.ReferencedConfiguration);
                            Debug.Print("        Component Path: " + swComp.GetPathName());
                        }
                        else
                        {
                            Debug.Print("    Could not get component.");
                        }
                    }
                }
            }
        }
    }
}
```

```

public void ProcessBomFeature(SldWorks swApp, ModelDoc2 swModel, BomFeature swBomFeat)
{
    Feature swFeat = default(Feature);
    object[] vTableArr = null;
    object vTable = null;
    string[] vConfigArray = null;
    object vConfig = null;
    string ConfigName = null;
    TableAnnotation swTable = default(TableAnnotation);
    object visibility = null;

    swFeat = swBomFeat.GetFeature();
    vTableArr = (object[])swBomFeat.GetTableAnnotations();

    foreach (TableAnnotation vTable_loopVariable in vTableArr)
    {
        vTable = vTable_loopVariable;
        swTable = (TableAnnotation)vTable;
        vConfigArray = (string[])swBomFeat.GetConfigurations(true, ref visibility);
        foreach (object vConfig_loopVariable in vConfigArray)
        {
            vConfig = vConfig_loopVariable;
            ConfigName = (string)vConfig;
            Debug.Print(" Component for Configuration: " + ConfigName);
            ProcessTableAnn(swApp, swModel, swTable, ConfigName);
        }
    }
}

public void Main()
{
    ModelDoc2 swModel = default(ModelDoc2);
    DrawingDoc swDraw = default(DrawingDoc);
    Feature swFeat = default(Feature);
    BomFeature swBomFeat = default(BomFeature);

    swModel = (ModelDoc2)swApp.ActiveDoc;
    swDraw = (DrawingDoc)swModel;
    swFeat = (Feature)swModel.FirstFeature();

    while ((swFeat != null))
    {
        if ("BomFeat" == swFeat.GetTypeName())
        {
            Debug.Print("Feature Name: " + swFeat.Name);
            swBomFeat = (BomFeature)swFeat.GetSpecificFeature2();
            ProcessBomFeature(swApp, swModel, swBomFeat);
        }
        swFeat = (Feature)swFeat.GetNextFeature();
    }
}

public SldWorks swApp;
}
}

```

Get Components in Each BOM Table Row Example (VB.NET)

This example shows how to get the components in each row of a BOM table annotation.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
' 2. Make a drawing from the assembly.
' 3. Select Insert > Tables > Bill of Materials.
' 4. Ensure that Parts only in BOM Type is selected.
' 5. Ensure that Display configurations of the same part as separate items
'     in Part Configuration Grouping is selected.
' 6. Click OK.
' 7. Click anywhere in the drawing to insert the BOM table.
'
```

```
' Postconditions:
' 1. Gets the Bill of Materials1 feature.
' 2. Gets the Default configuration.
' 3. Processes the BOM table for the Default configuration.
' 4. Examine the Immediate window.
'
```

```
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub ProcessTableAnn(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swTableAnn As TableAnnotation, ByVal ConfigName As String)
```

```
        Dim numRows As Long
        Dim J As Long
        Dim I As Long
        Dim ItemNumber As String = Nothing
        Dim PartNumber As String = Nothing
        Dim RowLocked As Boolean
        Dim RowHeight As Double
```

```
        Debug.Print("    Table Title: " & swTableAnn.Title)
```

```
        numRows = swTableAnn.RowCount
```

```
        Dim swBOMTableAnn As BomTableAnnotation
        swBOMTableAnn = swTableAnn
```

```
        For J = 0 To numRows - 1
            RowLocked = swTableAnn.GetLockRowHeight(J)
            RowHeight = swTableAnn.GetRowHeight(J)
            Debug.Print("    Row Number " & J & " (height = " & RowHeight & "; height locked = " & RowLocked & ")")
            Debug.Print("        Component Count: " & swBOMTableAnn.GetComponentsCount2(J, ConfigName, ItemNumber, PartNumber))
            Debug.Print("        Item Number: " & ItemNumber)
            Debug.Print("        Part Number: " & PartNumber)
```

```
            Dim vPtArr As Object
            Dim swComp As Object
            Dim pt As Object
```

```
            vPtArr = swBOMTableAnn.GetComponents2(J, ConfigName)
```

```
            If (Not vPtArr Is Nothing) Then
                For I = 0 To UBound(vPtArr)
                    pt = vPtArr(I)
                    swComp = pt
                    If Not swComp Is Nothing Then
                        Debug.Print("                Component Name: " & swComp.Name2)
                        Debug.Print("                Configuration Name: " & swComp.ReferencedConfiguration)
                        Debug.Print("                Component Path: " & swComp.GetPathName)
                    Else
                        Debug.Print("    Could not get component.")
                    End If
                Next
            End If
```

```
        Next
```

```
    End If
```

```
Next J
```

```
End Sub
```

```
Sub ProcessBomFeature(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swBomFeat As BomFeature)
```

```
    Dim swFeat As Feature
    Dim vTableArr As Object
    Dim vTable As Object
    Dim vConfigArray As Object
    Dim vConfig As Object
    Dim ConfigName As String
    Dim swTable As TableAnnotation
```

```
    swFeat = swBomFeat.GetFeature
    vTableArr = swBomFeat.GetTableAnnotations
```

```
    For Each vTable In vTableArr
        swTable = vTable
        vConfigArray = swBomFeat.GetConfigurations(True, True)
```

```

        For Each vConfig In vConfigArray
            ConfigName = vConfig
            Debug.Print(" Component for Configuration: " & ConfigName)
            ProcessTableAnn(swApp, swModel, swTable, ConfigName)
        Next vConfig
    Next vTable

End Sub

Sub main()

    Dim swModel As ModelDoc2
    Dim swDraw As DrawingDoc
    Dim swFeat As Feature
    Dim swBomFeat As BomFeature

    swModel = swApp.ActiveDoc
    swDraw = swModel
    swFeat = swModel.FirstFeature

    Do While Not swFeat Is Nothing
        If "BomFeat" = swFeat.GetTypeName Then
            Debug.Print("Feature Name: " & swFeat.Name)

            swBomFeat = swFeat.GetSpecificFeature2

            ProcessBomFeature(swApp, swModel, swBomFeat)
        End If
        swFeat = swFeat.GetNextFeature
    Loop
End Sub

Public swApp As SldWorks

End Class

```

Get Components in Each BOM Table Row Example (VBA)

This example shows how to get the components in each row of a BOM table annotation.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
' 2. Make a drawing from the assembly.
' 3. Select Insert > Tables > Bill of Materials.
' 4. Ensure that Parts only in BOM type is selected.
' 5. Ensure that Display configurations of the same part as separate items
'    in Part configuration Grouping is selected.
' 6. Click OK.
' 7. Click anywhere in the drawing to insert the BOM table.
'
' Postconditions:
' 1. Gets the Bill of Materials1 feature.
' 2. Gets the Default configuration.
' 3. Processes the BOM table for the Default configuration.
' 4. Examine the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----

```

Option Explicit

```
Sub ProcessTableAnn(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTableAnn As SldWorks.TableAnnotation, ConfigName As String)
```

```

    Dim nNumRow As Long
    Dim nNumColumn As Long
    Dim J As Long
    Dim I As Long
    Dim ItemNumber As String
    Dim PartNumber As String
    Dim RowLocked As Boolean
    Dim RowHeight As Double

    Debug.Print "    Table Title: " & swTableAnn.Title

    nNumRow = swTableAnn.RowCount

    Dim swBOMTableAnn As BomTableAnnotation
    Set swBOMTableAnn = swTableAnn

    For J = 0 To nNumRow - 1
        RowLocked = swTableAnn.GetLockRowHeight(J)
        RowHeight = swTableAnn.GetRowHeight(J)
        Debug.Print "    Row Number " & J & " (height = " & RowHeight & "; height locked = " & RowLocked & ")"
        Debug.Print "        Component Count: " & swBOMTableAnn.GetComponentsCount2(J, ConfigName, ItemNumber, PartNumber)
        Debug.Print "            Item Number: " & ItemNumber
        Debug.Print "            Part Number: " & PartNumber

        Dim vPtArr As Variant
        Dim swComp As Object
        Dim pt As Object
        Dim compPath As String

        vPtArr = swBOMTableAnn.GetComponents2(J, ConfigName)

        If (Not IsEmpty(vPtArr)) Then
            For I = 0 To UBound(vPtArr)
                Set pt = vPtArr(I)
                Set swComp = pt
                If Not swComp Is Nothing Then
                    Debug.Print "            Component Name: " & swComp.Name2
                    Debug.Print "            Configuration Name: " & swComp.ReferencedConfiguration
                    Debug.Print "            Component Path: " & swComp.GetPathName
                Else
                    Debug.Print "            Could not get component."
                End If
            Next
        End If
    Next J
End Sub

```

```
Sub ProcessBomFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swBomFeat As SldWorks.BomFeature)
```

```

    Dim swFeat As SldWorks.Feature
    Dim vTableArr As Variant
    Dim vTable As Variant
    Dim vConfigArray As Variant
    Dim vConfig As Variant
    Dim ConfigName As String
    Dim swTable As SldWorks.TableAnnotation

    Set swFeat = swBomFeat.GetFeature
    vTableArr = swBomFeat.GetTableAnnotations

    For Each vTable In vTableArr
        Set swTable = vTable
        vConfigArray = swBomFeat.GetConfigurations(True, True)
        For Each vConfig In vConfigArray
            ConfigName = vConfig
            Debug.Print "    Component for configuration: " & ConfigName
            ProcessTableAnn swApp, swModel, swTable, ConfigName
        Next
    Next
End Sub

```

```

        Next vConfig
    Next vTable
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swFeat As SldWorks.Feature
    Dim swBomFeat As SldWorks.BomFeature

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swFeat = swModel.FirstFeature

    Do While Not swFeat Is Nothing
        If "BomFeat" = swFeat.GetTypeName Then
            Debug.Print "Feature Name: " & swFeat.Name
            Set swBomFeat = swFeat.GetSpecificFeature2
            ProcessBomFeature swApp, swModel, swBomFeat
        End If
        Set swFeat = swFeat.GetNextFeature
    Loop
End Sub

```

Get Components' Properties in Drawing View Example (VBA)

This example shows how to get the components in a drawing view.

```
'-----
'
' Preconditions:
'     (1) Drawing document of an assembly is open.
'     (2) Drawing view is selected.
'
' Postconditions: None
'
'-----

Option Explicit
Public Enum swLineWeights_e
    swLW_NONE = -1
    swLW_THIN = 0
    swLW_NORMAL = 1
    swLW_THICK = 2
    swLW_THICK2 = 3
    swLW_THICK3 = 4
    swLW_THICK4 = 5
    swLW_THICK5 = 6
    swLW_THICK6 = 7
    swLW_NUMBER = 8
    swLW_LAYER = 9
End Enum

Public Enum swLineStyles_e
    swLineCONTINUOUS = 0
    swLineHIDDEN = 1
    swLinePHANTOM = 2
    swLineCHAIN = 3
    swLineCENTER = 4
    swLineSTITCH = 5
    swLineCHAINTHICK = 6
    swLineDEFAULT = 7
End Enum

' The different types of drawing views
Public Enum swDrawingViewTypes_e
    swDrawingSheet = 1
    swDrawingSectionView = 2
    swDrawingDetailView = 3
    swDrawingProjectedView = 4
    swDrawingAuxiliaryView = 5
    swDrawingStandardView = 6
    swDrawingNamedView = 7
    swDrawingRelativeView = 8
End Enum

Sub ProcessDrawingComponent _
(
    swDrawComp As SldWorks.DrawingComponent, _
    sPadStr As String _
)
    Dim vDrawCompChildArr As Variant
    Dim vDrawCompChild As Variant
    Dim swDrawCompChild As SldWorks.DrawingComponent

    ' Returns empty strings for root component
    Debug.Print sPadStr & "Name" = " & swDrawComp.component.Name2
    Debug.Print sPadStr & " File" = " & swDrawComp.component.GetPathName

    Debug.Print sPadStr & " IsRoot" = " & swDrawComp.IsRoot
    Debug.Print sPadStr & " Layer" = " & swDrawComp.Layer
```

```

Debug.Print sPadStr & " LayerOverride = " & swDrawComp.LayerOverride
Debug.Print sPadStr & " Style = " & swDrawComp.Style
Debug.Print sPadStr & " Visible = " & swDrawComp.Visible
Debug.Print sPadStr & " Width = " & swDrawComp.Width
Debug.Print ""

```

```

vDrawCompChildArr = swDrawComp.GetChildren
If Not IsEmpty(vDrawCompChildArr) Then
    For Each vDrawCompChild In vDrawCompChildArr
        Set swDrawCompChild = vDrawCompChild

        ProcessDrawingComponent swDrawCompChild, sPadStr + " "
    Next
End If
End Sub

```

```

Sub main()
    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swDraw           As SldWorks.DrawingDoc
    Dim swSelMgr          As SldWorks.SelectionMgr
    Dim swView            As SldWorks.View
    Dim swDrawComp        As SldWorks.DrawingComponent
    Dim bRet              As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject5(1)
    Set swDrawComp = swView.RootDrawingComponent

```

```

Debug.Print "File = " & swModel.GetPathName
Debug.Print " " & swView.Name & " [" & swView.Type & "]"

```

```

ProcessDrawingComponent swDrawComp, " "
End Sub

```

```

'-----

```

Get Configuration-specific Cut List Custom Properties Example (C#)

This example shows how to get configuration-specific cut list custom properties.

```
// =====
// Preconditions:
// 1. Open a part that has one or more configuration-specific cut lists with custom properties.
//    (Or ask SOLIDWORKS API Support to send you the models that work with this ICutListItem sample code.)
// 2. Open an Immediate/Output window.
//
// Postconditions: Inspect all the configurations' cut lists' custom properties in the Immediate/Output window.
// =====

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace ConfigSpecCutListCustomProps_CSharp
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        string[] vConfigNameArr;
        string vConfigName;
        object[] cutlistobj;
        CutListItem[] vcutlistItems;
        CutListItem vcutListItem;
        CutListItem cutListItem;
        CustomPropertyManager cusPropMgr;
        Configuration config;
        int lRetVal;
        object vPropNames;
        object vPropTypes;
        object vPropValues;
        object resolved;
        object linkProp;
        int i;
        int j;
        string propName;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            vConfigNameArr = (string[])swModel.GetConfigurationNames();
            foreach (var vConfigName in vConfigNameArr)
            {
                Debug.Print("Configuration: " + vConfigName);
                config = (Configuration)swModel.GetConfigurationByName(vConfigName);
                cutlistobj = (object[])config.GetCutListItems();

                int n;
```

```

for (n=0; n < cutlistobj.Length; n++)
{
    vcutlistItem = (CutListItem)cutlistobj[n];

    Debug.Print("Cut list item #" + n + " custom properties:");
    cusPropMgr = vcutlistItem.CustomPropertyManager;
    lRetVal = cusPropMgr.GetAll3(ref vPropNames, ref vPropTypes, ref vPropValues, ref resolved, ref linkProp);

    object[] propValues = (object[])vPropValues;
    string[] propNames = (string[])vPropNames;

    for (j = 0; j < lRetVal; j++)
    {
        Debug.Print(" " + propNames[j] + " " + propValues[j]);

    }

    i = i + 1;
}
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

Get Configuration-specific Cut List Custom Properties Example (VB.NET)

This example shows how to get configuration-specific cut list custom properties.

```
'=====
```

```
'Preconditions:
```

```
' 1. Open a part that has one or more configuration-specific cut lists with custom properties.
'    (Or ask SOLIDWORKS API Support to send you the models that work with this ICutListItem sample code.)
' 2. Open an Immediate/Output window.
```

```
'Postconditions: Inspect all the configurations' cut lists' custom properties in the Immediate/Output window.
```

```
'=====
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim vConfigNameArr() As Object
    Dim vConfigName As String
    Dim vcutlistItems() As Object
    Dim vcutListItem As CutListItem
    Dim cutListItem As CutListItem
    Dim cusPropMgr As CustomPropertyManager
    Dim config As Configuration
    Dim lRetVal As Integer
    Dim vPropNames() As String
    Dim vPropTypes() As Integer
    Dim vPropValues() As String
    Dim resolved() As Integer
    Dim linkProp() As Integer
    Dim i As Integer
    Dim j As Integer
    Dim propName As String

    Public Sub main()

        swModel = swApp.ActiveDoc

        vConfigNameArr = swModel.GetConfigurationNames()
        For Each vConfigName In vConfigNameArr
            Debug.Print("Configuration: " & vConfigName)
            config = swModel.GetConfigurationByName(vConfigName)
            vcutlistItems = config.GetCutListItems()
            i = 1
            For Each vcutListItem In vcutlistItems
                Debug.Print("Cut list item #" & i & " custom properties:")
                cutListItem = vcutListItem
                cusPropMgr = cutListItem.CustomPropertyManager()
                lRetVal = cusPropMgr.GetAll3(vPropNames, vPropTypes, vPropValues, resolved, linkProp)
                j = 0
                For Each propName In vPropNames
                    Debug.Print(" " & propName & " " & vPropValues(j))
                    j = j + 1
                Next
                i = i + 1
            Next
        Next
    End Sub
End Class
```

Next

End Sub

''' <summary>

''' The SldWorks swApp variable is pre-assigned for you.

''' </summary>

Public swApp As SldWorks

End Class

Get Configuration Parameters Example (C++ COM)

This example shows how to get the parameters for the specified configuration.

```
'-----
'
' Preconditions: Part is open and has a configuration named aaa.
'
' Postconditions: None
'
'-----

#define USING_DEBUG_SOLIDWORKS    TRUE
#ifdef _DEBUG && USING_DEBUG_SOLIDWORKS
    #define AFX_MANAGE_STATE_ADDIN    AFX_MANAGE_STATE(AfxGetAppModuleState());
#else
    #define AFX_MANAGE_STATE_ADDIN    AFX_MANAGE_STATE(AfxGetStaticModuleState());
#endif

// -----
STDMETHODIMP CAPITestATL::API_Test(void)
{
    AFX_MANAGE_STATE_ADDIN;
    HRESULT          hr = S_OK;
    CComPtr <ISldWorks>    pSldWorks;
    CComPtr <IModelDoc2>    pModel;
    CComPtr <IConfigurationManager>    pCfgMgr;
    CComBSTR          sCfgName(_T("aaa"));
    long              NumParams = 0;
    long              i = 0;
    BSTR              *vNames = NULL;
    BSTR              *vValues = NULL;
    VARIANT_BOOL      bret = VARIANT_FALSE;
    try
    {
        // Connect to SOLIDWORKS Design
        ASSERT(m_iSldWorks);
        hr = m_iSldWorks->get_IActiveDoc2(&pModel);
        ASSERT(pModel);
        hr = pModel->get_IConfigurationManager(&pCfgMgr);
        ASSERT(pCfgMgr);
        hr = pCfgMgr->GetConfigurationParamsCount(sCfgName, &NumParams);
        ASSERT(NumParams > 0);
        vNames = new BSTR[NumParams]; ASSERT(vNames);
        vValues = new BSTR[NumParams]; ASSERT(vValues);
        hr = pCfgMgr->IGetConfigurationParams(sCfgName, NumParams, vNames, vValues, &bret);
        ASSERT(bret);
        for (i = 0; i < NumParams; i++)
        {
            CComBSTR      sName ;
            CComBSTR      sValue;

            sName.Attach(vNames[i]);
            sValue.Attach(vValues[i]);
            OutputDebugString(sName);
            OutputDebugString(_T("\n"));
            OutputDebugString(sValue);
            OutputDebugString(_T("\n"));
        }
        delete [] vNames ;
        delete [] vValues ;
    }
    catch (...)
    {
        DebugBreak();
    }
    delete [] vNames ;
    delete [] vValues ;
    return S_OK;
}
```


Get Configurations Referenced in View Example (C#)

This example shows how to get the names and persistent reference IDs of the configurations referenced in each drawing view in the first drawing sheet.

```
//-----
// Preconditions:
// 1. Open a drawing document with multiple
//    drawing views and multiple referenced
//    configurations in the first drawing sheet
//    in the drawing.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the drawing views in the first
//    drawing sheet and gets each drawing view's:
//    * name
//    * referenced model name
//    * referenced configuration name and
//    * persistent reference ID
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            View swView = default(View);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;
            Debug.Print("File = " + swModel.GetPathName());

            // First drawing view is actually the first drawing sheet,
            // so skip getting model name and configuration from
            // the drawing sheet
            swView = (View)swDraw.GetFirstView();
            // Get first drawing view in first drawing sheet
            swView = (View)swView.GetNextView();
            while ((swView != null))
            {
                Debug.Print("  Drawing view = " + swView.Name);
                Debug.Print("    Referenced model name = " + swView.GetReferencedModelName());
                Debug.Print("    Referenced configuration name = " + swView.ReferencedConfiguration);
                Debug.Print("    Referenced configuration persistent reference ID = " + swView.ReferencedConfigurationID);
                //Get next drawing view
                swView = (View)swView.GetNextView();
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Configurations Referenced in View Example (VBA)

This example shows how to get the names and persistent reference IDs of the configurations referenced in each drawing view in the first drawing sheet.

```
'-----
' Preconditions:
' 1. Open a drawing document with multiple
'    drawing views and multiple referenced
'    configurations in the first drawing sheet
'    in the drawing.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the drawing views in the first
'    drawing sheet and gets each drawing view's:
'    * name
'    * referenced model name
'    * referenced configuration name and
'    * persistent reference ID
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    Debug.Print "File = " & swModel.GetPathName

    ' First drawing view is actually the first drawing sheet,
    ' so skip getting model name and configuration from
    ' the drawing sheet
    Set swView = swDraw.GetFirstView
    ' Get first drawing view in first drawing sheet
    Set swView = swView.GetNextView
    Do While Not swView Is Nothing
        Debug.Print "    Drawing view = " & swView.Name
        Debug.Print "        Referenced model name = " & swView.GetReferencedModelName
        Debug.Print "        Referenced configuration name = " & swView.ReferencedConfiguration
        Debug.Print "        Referenced configuration persistent reference ID = " & swView.ReferencedConfigurationID
        'Get next drawing view
        Set swView = swView.GetNextView
    Loop

End Sub
```

Get Configurations Referenced in View Example (VB.NET)

This example shows how to get the names and persistent reference IDs of the configurations referenced in each drawing view in the first drawing sheet.

```
'-----
' Preconditions:
' 1. Open a drawing document with multiple
'    drawing views and multiple referenced
'    configurations in the first drawing sheet
'    in the drawing.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the drawing views in the first
'    drawing sheet and gets each drawing view's:
'    * name
'    * referenced model name
'    * referenced configuration name and
'    persistent reference ID
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swView As View

        swModel = swApp.ActiveDoc
        swDraw = swModel
        Debug.Print("File = " & swModel.GetPathName)

        ' First drawing view is actually the first drawing sheet,
        ' so skip getting model name and configuration from
        ' the drawing sheet
        swView = swDraw.GetFirstView
        ' Get first drawing view in first drawing sheet
        swView = swView.GetNextView
        Do While Not swView Is Nothing
            Debug.Print("  Drawing view = " & swView.Name)
            Debug.Print("    Referenced model = " & swView.GetReferencedModelName)
            Debug.Print("    Referenced configuration name = " & swView.ReferencedConfiguration)
            Debug.Print("    Referenced configuration persistent reference ID = " & swView.ReferencedConfigurationID)
            'Get next drawing view
            swView = swView.GetNextView
        Loop

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Contents of FeatureFolder Example (C#)

This example shows how to obtain the contents of feature folders.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Right-click a feature in the FeatureManager design tree.
// 3. Select Add to New Folder, which adds the feature to
//    the new folder.
// 4. Open the Immediate Window.
//
// Postconditions: Inspect the Immediate Window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace FeatureFolder_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            IFeatureFolder swFeatFolder = default(IFeatureFolder);
            Feature swFtrFolder = default(Feature);
            string FeatType = null;
            string FeatTypeName = null;
            object[] Features = null;
            int i = 0;

            Feature[] featureArr = new Feature[3];

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swFeatMgr = swModel.FeatureManager;

            swFeat = (Feature)swModel.FirstFeature();
            while ((swFeat != null))
            {
                FeatType = swFeat.Name;
                FeatTypeName = swFeat.GetTypeName2();

                if ((FeatTypeName == "FtrFolder" & !FeatType.Contains("EndTag")))
                {
                    Debug.Print(FeatType + " [" + FeatTypeName + "]");
                    swFeatFolder = (IFeatureFolder)swFeat.GetSpecificFeature2();
                    Features = (object[])swFeatFolder.GetFeatures();
                    Debug.Print(" Number of Features: " + swFeatFolder.GetFeatureCount());
                    for (i = 0; i <= (swFeatFolder.GetFeatureCount() - 1); i++)
                    {
                        swFtrFolder = (Feature)Features[i];
                        Debug.Print(" Name of feature: " + swFtrFolder.Name);
                        Debug.Print(" Type of feature: " + swFtrFolder.GetTypeName2());
                    }
                }
                swFeat = (Feature)swFeat.GetNextFeature();
            }
        }
    }
    public SldWorks swApp;
}
}
```

Get Contents of FeatureFolder Example (VBA)

This example shows how to obtain the contents of feature folders.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Right-click a feature in the FeatureManager design tree.
' 3. Select Add to New Folder, which moves the feature to
'    the new folder.
' 4. Open the Immediate Window.
'
' Postconditions: Inspect the Immediate Window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swFeat As SldWorks.Feature
    Dim swFeatFolder As SldWorks.FeatureFolder
    Dim swFtrFolder As SldWorks.Feature
    Dim FeatType As String
    Dim FeatTypeName As String
    Dim NbrOfFeatures As Long
    Dim Features As Variant
    Dim i As Long
    Dim featureArr(2) As SldWorks.Feature
    Dim featureObj As Variant

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager

    Set swFeat = swModel.FirstFeature
    Do While Not swFeat Is Nothing
        FeatType = swFeat.Name
        FeatTypeName = swFeat.GetTypeName2
        Dim temp As Long
        temp = InStr(1, FeatType, "EndTag", vbTextCompare)
        If (FeatTypeName = "FtrFolder" And temp = 0) Then
            Debug.Print FeatType & " [" & FeatTypeName & "]"
            Set swFeatFolder = swFeat.GetSpecificFeature2
            Features = swFeatFolder.GetFeatures
            Debug.Print "    Number of Features: " & swFeatFolder.GetFeatureCount
            For i = 0 To (swFeatFolder.GetFeatureCount - 1)
                Set swFtrFolder = Features(i)
                Debug.Print "    Name of feature: " & swFtrFolder.Name
                Debug.Print "    Type of feature: " & swFtrFolder.GetTypeName2
            Next i
        End If
        Set swFeat = swFeat.GetNextFeature
    Loop

End Sub

```

Get Contents of FeatureFolder Example (VB.NET)

This example shows how to obtain the contents of feature folders.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Right-click a feature in the FeatureManager design tree.
' 3. Select Add to New Folder, which adds the feature to
'    the new folder.
' 4. Open the Immediate Window.
'-----
' Postconditions: Inspect the Immediate Window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim swFeatFolder As IFeatureFolder
        Dim swFtrFolder As Feature
        Dim FeatType As String
        Dim FeatTypeName As String
        Dim Features As Object
        Dim i As Long

        Dim featureArr(2) As Feature

        swModel = swApp.ActiveDoc
        swFeatMgr = swModel.FeatureManager

        swFeat = swModel.FirstFeature
        Do While Not swFeat Is Nothing
            FeatType = swFeat.Name
            FeatTypeName = swFeat.GetTypeName2
            Dim temp As Long
            temp = InStr(1, FeatType, "EndTag", vbTextCompare)

            If (FeatTypeName = "FtrFolder" And temp = 0) Then
                Debug.Print(FeatType & " [" & FeatTypeName & "]")
                swFeatFolder = swFeat.GetSpecificFeature2
                Features = swFeatFolder.GetFeatures
                Debug.Print("    Number of Features: " & swFeatFolder.GetFeatureCount)
                For i = 0 To (swFeatFolder.GetFeatureCount - 1)
                    swFtrFolder = Features(i)
                    Debug.Print("        Name of feature: " & swFtrFolder.Name)
                    Debug.Print("        Type of feature: " & swFtrFolder.GetTypeName2)
                Next i
            End If
            swFeat = swFeat.GetNextFeature
        Loop

    End Sub

    Public swApp As SldWorks

End Class

```

Get Coordinate System Transform Example (VBA)

This example shows how to get the default coordinate system transform for the selected feature.

```
'-----
'
' Preconditions: Model document is open and a feature is selected.
'
' Postconditions: None
'
'-----

Option Explicit
Public Enum swUserPreferenceStringValue_e
    swFileSaveAsCoordinateSystem = 16
End Enum

Sub main()
    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swDocExt         As SldWorks.ModelDocExtension
    Dim swSelMgr         As SldWorks.SelectionMgr
    Dim swFeat           As SldWorks.Feature
    Dim swXform          As SldWorks.MathTransform
    Dim sAxisName        As String
    Dim bRet             As Boolean
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject5(1)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Current coordinate system = " & swModel.GetUserPreferenceStringValue(swFileSaveAsCoordinateSystem)
    Debug.Print ""

    sAxisName = swFeat.Name
    Set swXform = swDocExt.GetCoordinateSystemTransformByName(sAxisName)
    Debug.Print " " & sAxisName
    Debug.Print " Origin          = (" & -1# * swXform.ArrayData(9) * 1000# & ", " & -1# * swXform.ArrayData(10) * 1000# & ", " & -1# * swXform.ArrayData(11) * 1000# & ") mm"
    Debug.Print " Rotational sub-matrix 1  = (" & swXform.ArrayData(0) & ", " & swXform.ArrayData(1) & ", " & swXform.ArrayData(2) & ")"
    Debug.Print " Rotational sub-matrix 2  = (" & swXform.ArrayData(3) & ", " & swXform.ArrayData(4) & ", " & swXform.ArrayData(5) & ")"
    Debug.Print " Rotational sub-matrix 3  = (" & swXform.ArrayData(6) & ", " & swXform.ArrayData(7) & ", " & swXform.ArrayData(8) & ")"
    Debug.Print " Translation vector      = (" & swXform.ArrayData(9) * 1000# & ", " & swXform.ArrayData(10) * 1000# & ", " & swXform.ArrayData(11) * 1000# & ") mm"
    Debug.Print " Scale                  = " & swXform.ArrayData(12)
End Sub

'-----
```

Get Coordinates of the Plane's Bounding Box Example (C#)

This example shows how to get top-left and bottom-right coordinates for a reference plane's bounding box.

```
//-----
// Preconditions:
// 1. Verify that the specified part document template
// exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Inserts a reference plane.
// 3. Gets the top-left and bottom-right coordinates
// of the reference plane's bounding box.
// 4. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatureMgr = default(FeatureManager);
            RefPlane swRefPlane = default(RefPlane);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MathPoint swMathPoint = default(MathPoint);
            bool status = false;
            object[] mathPoints = null;
            double[] arrData = null;
            int i = 0;

            //Create reference plane
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swRefPlane = (RefPlane)swFeatureMgr.InsertRefPlane(8, 0.0889, 0, 0, 0, 0);

            //Get top-left and bottom-right coordinates
            //of the reference plane's bounding box
            mathPoints = (object[])swRefPlane.BoundingBox;
            //Two (2) MathPoint objects are always returned
            for (i = 0; i <= (mathPoints.Length - 1); i++)
            {
                swMathPoint = (MathPoint)mathPoints[i];
                arrData = (double[])swMathPoint.ArrayData;
                Debug.Print(" Points x, y, z = " + arrData[0] + ", " + arrData[1] + ", " + arrData[2]);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Coordinates of the Plane's Bounding Box Example (VBA)

This example shows how to get top-left and bottom-right coordinates for a reference plane's bounding box.

```
'-----
' Preconditions:
' 1. Verify that the specified part document template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Inserts a reference plane.
' 3. Gets the top-left and bottom-right coordinates
'    of the reference plane's bounding box.
' 4. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swRefPlane As SldWorks.RefPlane
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim status As Boolean
Dim mathPoints As Variant
Dim arrData As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Create reference plane
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swRefPlane = swFeatureMgr.InsertRefPlane(8, 0.0889, 0, 0, 0, 0)

    'Get top-left and bottom-right coordinates
    'of the reference plane's bounding box
    mathPoints = swRefPlane.BoundingBox 'Two (2) MathPoint objects are always returned
    For i = 0 To UBound(mathPoints)
        arrData = mathPoints(i).arrayData
        Debug.Print " Points x, y, z = " & arrData(0) & ", " & arrData(1) & ", " & arrData(2)
    Next i

End Sub
```

Get Coordinates of the Plane's Bounding Box Example (VB.NET)

This example shows how to get top-left and bottom-right coordinates for a reference plane's bounding box.

```

-----
Preconditions:
1. Verify that the specified part document template
   exists.
2. Open the Immediate window.
Postconditions:
1. Opens a new part document.
2. Inserts a reference plane.
3. Gets the top-left and bottom-right coordinates
   of the reference plane's bounding box.
4. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatureMgr As FeatureManager
        Dim swRefPlane As RefPlane
        Dim swModelDocExt As ModelDocExtension
        Dim status As Boolean
        Dim mathPoints() As Object
        Dim arrData() As Double
        Dim i As Integer

        'Create reference plane
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swRefPlane = swFeatureMgr.InsertRefPlane(8, 0.0889, 0, 0, 0, 0)

        'Get top-left and bottom-right coordinates
        'of the reference plane's bounding box
        mathPoints = swRefPlane.BoundingBox 'Two (2) MathPoint objects are always returned
        For i = 0 To UBound(mathPoints)
            arrData = mathPoints(i).arrayData
            Debug.Print(" Points X, y, z = " & arrData(0) & ", " & arrData(1) & ", " & arrData(2))
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Core Feature Data Example (C#)

This example shows how to get the data for a core feature.

```
//-----
// Preconditions:
// 1. Open a model document that contains a core feature.
// 2. Open the Immediate window.
// 3. Select the core feature in the FeatureManager design tree.
//
// Postconditions:
// 1. Prints the core feature data to the Immediate window.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CoreFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Feature swFeat = default(Feature);
            CoreFeatureData swCoreFeat = default(CoreFeatureData);
            bool b = false;
            string nam = null;
            bool cap = false;
            double d1 = 0;
            double d2 = 0;
            double ang = 0;
            bool useDr = false;
            bool Drout = false;
            bool rev = false;
            int typ1 = 0;
            int typ2 = 0;
            object dir1 = null;
            object dir2 = null;
            int e1 = 0;
            int e2 = 0;
            int ct = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swCoreFeat = (CoreFeatureData)swFeat.GetDefinition();

            //Roll back to the feature before the core feature
            b = swCoreFeat.AccessSelections(swModel, null);

            //Get the bounding sketch of the core feature
            swFeat = swCoreFeat.BoundingSketch;
            nam = swFeat.Name;
            Debug.Print("Name of bounding sketch = " + nam);
            cap = swCoreFeat.CapEnds;
            Debug.Print("Are ends capped? " + cap);
            d1 = swCoreFeat.get_Depth(0);
            Debug.Print("Depth along extraction direction = " + d1);
            d2 = swCoreFeat.get_Depth(1);
            Debug.Print("Depth away from extraction direction = " + d2);
            useDr = swCoreFeat.UseDraft;
            Debug.Print("Drafted? " + useDr);
            if (useDr)
            {
                ang = swCoreFeat.DraftAngle;
                Debug.Print("Angle of draft = " + ang);
                Drout = swCoreFeat.DraftOutward;
                Debug.Print("Drafted outward? " + Drout);
            }
            e1 = swCoreFeat.get_EndCondition(0);
            Debug.Print("End condition along extraction = " + e1);
            e2 = swCoreFeat.get_EndCondition(1);
            Debug.Print("End condition away from extraction = " + e2);
            rev = swCoreFeat.ReverseDirection;
            Debug.Print("Direction of extraction reversed? " + rev);
            ct = swCoreFeat.GetExtractionDirection(out typ1, out dir1, out typ2, out dir2);
            Debug.Print("Number of entities that define extraction = " + ct);

            //Roll forward
            swCoreFeat.ReleaseSelectionAccess();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Core Feature Data Example (VBA)

This example shows how to get the data for a core feature.

```
'-----
' Preconditions:
' 1. Open a model document that contains a core feature.
' 2. Open the Immediate window.
' 3. Select the core feature in the FeatureManager design tree.
'
' Postconditions:
' 1. Prints the core feature data to the Immediate window.
' 2. Examine the Immediate window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As Feature
Dim swCoreFeat As SldWorks.CoreFeatureData
Dim b As Boolean
Dim nam As String
Dim cap As Boolean
Dim d1 As Double
Dim d2 As Double
Dim ang As Double
Dim useDr As Boolean
Dim Drout As Boolean
Dim rev As Boolean
Dim typ1 As Long
Dim typ2 As Long
Dim dir1 As Object
Dim dir2 As Object
Dim e1 As Long
Dim e2 As Long
Dim ct As Long
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swCoreFeat = swFeat.GetDefinition

'Roll back to the feature before the core feature
b = swCoreFeat.AccessSelections(swModel, Nothing)

'Get the bounding sketch of the core feature
Set swFeat = swCoreFeat.BoundingSketch
nam = swFeat.Name
Debug.Print "Name of bounding sketch = " & nam
cap = swCoreFeat.CapEnds
Debug.Print "Are ends capped? " & cap
d1 = swCoreFeat.Depth(0)
Debug.Print "Depth along extraction direction = " & d1
d2 = swCoreFeat.Depth(1)
Debug.Print "Depth away from extraction direction = " & d2
useDr = swCoreFeat.UseDraft
Debug.Print "Drafted? " & useDr
If useDr Then
    ang = swCoreFeat.DraftAngle
    Debug.Print "Angle of draft = " & ang
    Drout = swCoreFeat.DraftOutward
    Debug.Print "Drafted outward? = " & Drout
End If
e1 = swCoreFeat.EndCondition(0)
Debug.Print "End condition along extraction = " & e1
e2 = swCoreFeat.EndCondition(1)
Debug.Print "End condition away from extraction = " & e2
rev = swCoreFeat.ReverseDirection
Debug.Print "Direction of extraction reversed? " & rev
ct = swCoreFeat.GetExtractionDirection(typ1, dir1, typ2, dir2)
Debug.Print "Number of entities that define extraction = " & ct

'Roll forward
swCoreFeat.ReleaseSelectionAccess
```

End Sub